

Chương 18: Bản Chất Quy Hoạch Động (Dynamic Programming)

[THUẬT NGỮ KHOA HỌC] Quy Hoạch Động & Không Gian Trạng Thái

Từ nguyên học: Thuật ngữ *Dynamic Programming* được nhà toán học Richard Bellman đặt ra vào thập niên 1950 để mô tả quá trình ra quyết định đa giai đoạn tối ưu tuần tự.

Mô hình trực giác: Quy hoạch động bản chất là phép đệ quy có ghi nhớ và triệt tiêu tính dư thừa. Khi một bài toán lớn có thể phân rã thành các bài toán con trùng lặp (Overlapping Subproblems) và nghiệm tối ưu của bài toán lớn được kiến tạo từ nghiệm tối ưu của các bài toán con (Optimal Substructure), ta lưu nghiệm của bài toán con vào một mảng trạng thái (State Table) để không bao giờ phải tính lại lần thứ hai.

18.1 Hai Điều Kiện Cốt Lõi Của Quy Hoạch Động

Một bài toán chỉ có thể giải bằng Quy hoạch động khi thỏa mãn đồng thời hai thuộc tính toán học:

- Cấu trúc con tối ưu (Optimal Substructure):** Lời giải tối ưu của bài toán kích thước N chứa bên trong nó lời giải tối ưu của các bài toán con kích thước $k < N$.
- Các bài toán con gối nhau (Overlapping Subproblems):** Cây gọi đệ quy thuần túy sẽ lặp đi lặp lại việc tính toán cùng một trạng thái nhiều lần (dẫn tới độ phức tạp hàm mũ $O(2^N)$ nếu không ghi nhớ).

18.2 Top-Down (Memoization) Đối Đầu Bottom-Up (Tabulation)

Có hai trường phái cài đặt thuật toán quy hoạch động:

1. Top-Down (Đệ Quy Nhớ - Memoization)

- Đi từ bài toán lớn N xuống các bài toán con nhỏ hơn 0, 1.
- Sử dụng mảng nhớ `memo[]` khởi tạo giá trị -1 .
- Ưu điểm:** Chỉ tính các trạng thái thực sự cần thiết trên cây gọi.
- Nhược điểm:** Tốn chi phí khung ngăn xếp hàm (Call Stack Overhead), nguy cơ tràn bộ nhớ stack (Stack Overflow) nếu N lớn.

2. Bottom-Up (Lập Bảng - Tabulation)

- Đi từ bài toán con cơ sở (0, 1) tính dần lên bài toán lớn N .
- Sử dụng vòng lặp tuần tự (`for i = 1 .. N`).
- Ưu điểm:** Tốc độ cực nhanh, không tốn stack, dễ tối ưu hóa bộ nhớ cuộn (Space Optimization).
- Nhược điểm:** Phải xác định thứ tự tô-pô tính toán các trạng thái trước khi viết vòng lặp.

[TIP] Quy tắc vàng trong Lập trình thi đấu

Trong các kỳ thi CP, luôn ưu tiên phương pháp **Bottom-Up Tabulation** vì nó loại bỏ hoàn toàn nguy cơ Stack Overflow và tận dụng tối đa cơ chế nạp bộ nhớ đệm CPU Cache Lines (L1/L2 Cache) nhờ việc truy xuất mảng liên tục trên RAM.

18.3 Thiết Kế Bảng Trạng Thái & Công Thức Chuyển Trạng Thái

Để xây dựng một thuật toán DP Bottom-Up hoàn chỉnh, ta tuân thủ 4 bước nghiêm ngặt:

- Định nghĩa trạng thái:** Ý nghĩa rõ ràng của $dp[i]$ (ví dụ: chi phí nhỏ nhất để chạm tới bậc i).
- Trạng thái cơ sở (Base Cases):** Giá trị khởi tạo không cần tính toán (ví dụ: $dp[1] = C_1, dp[2] = C_2$).
- Công thức chuyển trạng thái (State Transition):** Mối quan hệ truy hồi giữa $dp[i]$ và các trạng thái trước nó.

$$dp[i] = C_i + \min(dp[i-1], dp[i-2])$$

- Thứ tự tính toán & Đáp án cuối cùng:** Vòng lặp tăng dần từ 3 đến N , đáp án là $dp[N]$.

[MẸ XẾ CÚ PHÁP C++14]	
<pre>for (int i = 3; i ≤ n; ++i) { dp[i] = c[i] + std::min(dp[i-1], dp[i-2]); }</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<pre>for (int i = 3; i ≤ n; ++i)</pre>	Thứ tự tô-pô tuần tự đảm bảo $dp[i-1]$ và $dp[i-2]$ đã được tính tối ưu.
<pre>std::min(dp[i-1], dp[i-2])</pre>	Lựa chọn quyết định tối ưu giữa bước 1 bậc hoặc bước 2 bậc.
<pre>c[i] +</pre>	Cộng thêm chi phí dẫm lên bậc hiện tại.
<pre>dp[i] =</pre>	Lưu kết quả tối ưu vào bảng trạng thái để phục vụ các bước tương lai.

18.4 Bảng Chạy Bàn Trên Giấy: Mảng Chi Phí $C = [10, 15, 20, 10]$ Với $N = 4$

Bước	Bậc i	Chi phí $C[i]$	Lựa chọn trước đó	$dp[i]$ (Chi phí min)	$ways[i]$
Cơ sở	1	10	Khởi điểm tại 1	10	1
Cơ sở	2	15	Khởi điểm tại 2	15	1
3	3	20	$\min(dp[1]=10, dp[2]=15) \rightarrow$ chọn 1	$20 + 10 = 30$	$ways[1] = 1$
4	4	10	$\min(dp[2]=15, dp[3]=30) \rightarrow$ chọn 2	$10 + 15 = 25$	$ways[2] = 1$
Kết thúc	4	-	-	*Chi phí tối thiểu = 25*	*Số cách = 1*

18.5 Bài Tập Tiêu Chuẩn

**Bài toán: Bài 18.1 - Bản Chất Quy Hoạch Động: Bậc
Thang Chi Phí Tối Thiểu & Đếm Phương Án**

(Nguồn: Nền tảng C++14 - Quy hoạch động căn bản)

Có một cầu thang gồm N bậc ($1 \leq N \leq 30$). Chi phí khi dẫm lên bậc i là C_i ($1 \leq C_i \leq 1000$). Quy tắc di chuyển:

- Em có thể chọn bắt đầu dẫm lên bậc 1 (chi phí khởi điểm là C_1) hoặc bắt đầu dẫm lên bậc 2 (chi phí khởi điểm là C_2).
- Từ bậc i , mỗi bước có thể nhảy lên bậc $i + 1$ hoặc bậc $i + 2$.
- Đích đến là phải kết thúc chính xác tại bậc N .

Em hãy tính toán:

1. Tổng chi phí nhỏ nhất để đạt tới bậc N (`min_cost`).
2. Số lượng phương án di chuyển khác nhau đạt được đúng chi phí nhỏ nhất đó (`ways`).

Đầu vào (Input): Dòng 1 ghi số nguyên N . Dòng 2 ghi N số nguyên dương $C_1, C_2, \dots, C_N$.

Đầu ra (Output): In ra hai số nguyên `min_cost` `ways` cách nhau bởi khoảng trắng.

Ví dụ mẫu:

Đầu vào (Input):

Đầu ra (Output):

4
10 15 20 10

25 1

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	4 10 15 20 10	
2	Warm-up	5 1 2 1 1 3	
3	Warm-up	3 5 10 20	
4	Edge case	1 50	
5	Edge case	2 100 200	
6	Edge case	2 500 500	
7	Trick case	6 2 2 2 2 2 2	
8	Trick case	6 1 1 1 1 1 1	
9	Mental challenge	7 3 2 4 1 5 2 6	

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
10	Mental challenge	6 3 3 1 1 3 3	[.....]

Chương 19: Các Bài Toán Quy Hoạch Động Kinh Điển

[THUẬT NGỮ KHOA HỌC] Khuôn Mẫu DP Kinh Điển & Kỹ Thuật Nén Không Gian Trạng Thái

Từ nguyên học: Các bài toán như Knapsack, LIS, LCS được nghiên cứu sâu rộng từ thập niên 1960-1970 trong lý thuyết tối ưu hóa tổ hợp và tin sinh học (thuật toán Needleman-Wunsch).

Mô hình trực giác: Quy hoạch động trong lập trình thi đấu thường xoay quanh 4 mẫu hình nền tảng: Dây con tăng dài nhất (LIS), Chuỗi con chung dài nhất (LCS), Bài toán Cái túi (Knapsack), và Lưới tọa độ 2D (Grid DP). Nắm vững 4 khuôn mẫu này giúp ta phân loại và ánh xạ hầu hết các đề thi HSG/VNOI về dạng bài toán cốt lõi.

19.1 Bốn Khuôn Mẫu Quy Hoạch Động Nền Tảng

1. LIS (Longest Increasing Subsequence)

- $dp[i]$: Độ dài LIS kết thúc tại phần tử $A[i]$.
- Công thức: $dp[i] = 1 + \max_{j < i, A[j] < A[i]} dp[j]$.
- Độ phức tạp: $O(N^2)$.
- Nâng cấp $O(N \log N)$ bằng mảng `tail[]` và `std::lower_bound`.

2. LCS (Longest Common Subsequence)

- $dp[i][j]$: Độ dài LCS của tiền tố $S[1..i]$ và $T[1..j]$.
- Nếu $S[i] == T[j]$: $dp[i][j] = dp[i-1][j-1] + 1$.
- Nếu $S[i] \neq T[j]$: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.
- Độ phức tạp: $O(|S| \times |T|)$.

3. Knapsack 0/1 & Unbounded

- $dp[w]$: Giá trị cực đại khi sức chứa balo là w .
- 0/1 Knapsack:** Duyệt w từ W ngược về w_i (mỗi vật chọn tối đa 1 lần).
- Unbounded Knapsack:** Duyệt w từ w_i xuôi đến W (mỗi vật chọn vô hạn lần).

4. Grid DP (Lưới 2D)

- $dp[r][c]$: Số cách hoặc chi phí tối ưu từ ô $(1, 1)$ đến ô (r, c) .
- Chỉ di chuyển sang phải hoặc xuống dưới:

$$dp[r][c] = cost[r][c] + \min(dp[r-1][c], dp[r][c-1])$$

19.2 Chi Tiết Bài Toán Balo 0/1 (0/1 Knapsack)

Cho N vật phẩm, vật i có trọng lượng w_i và giá trị v_i . Balo có sức chứa tối đa W . Bảng trạng thái 2D: $dp[i][w]$ là giá trị lớn nhất khi xét i vật đầu tiên với sức chứa w .

Công thức chuyển trạng thái:

$$dp[i][w] = \begin{cases} dp[i-1][w] & \text{nếu } w < w_i \\ \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) & \text{nếu } w \geq w_i \end{cases}$$

[MÔ XẼ CÚ PHÁP C++14]

```
for (int cap = W; cap ≥ w[i]; --cap) { dp[cap] = std::max(dp[cap], dp[cap - w[i]] + v[i]); }
```

Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>for (int cap = W; cap ≥ w[i]; --cap)</code>	Duyệt ngược từ W về w[i] ngăn không cho một vật phẩm bị dùng 2 lần trong cùng một vòng lặp.
<code>dp[cap - w[i]]</code>	Truy xuất trạng thái của vòng lặp i-1 trước đó khi dung lượng còn lại là cap - w[i].
<code>+ v[i]</code>	Cộng thêm giá trị của vật phẩm i khi quyết định đưa vào balo.
<code>std::max(dp[cap], ...)</code>	So sánh giữa quyết định không chọn vật i và quyết định chọn vật i.

[DANGER] Cạm bẫy duyệt xuôi trong 0/1 Knapsack

Nếu viết `for (int cap = w[i]; cap ≤ W; ++cap)`, giá trị `dp[cap - w[i]]` đã bị cập nhật bởi chính vật phẩm i ở các bước trước đó trong cùng vòng lặp. Điều này biến bài toán 0/1 thành bài toán Unbounded Knapsack (cho phép chọn một vật nhiều lần)!

19.3 Bảng Chạy Bàn Trên Giấy: Balo $W = 7$ Với 4 Vật Phẩm

Các vật phẩm: 1 : ($w = 2, v = 3$), 2 : ($w = 3, v = 4$), 3 : ($w = 4, v = 5$), 4 : ($w = 5, v = 6$).

Xét vật i	Trọng lượng w	Giá trị v	Dung lượng cap = 2..4	Dung lượng cap = 5..7	Quyết định tối ưu
Khởi tạo	-	-	0, 0, 0	0, 0, 0	Mọi <code>dp[w] = 0</code>
Vật 1	2	3	<code>dp[2..3]=3, dp[4]=3</code>	<code>dp[5..7] = 3</code>	Chọn vật 1 nếu <code>cap ≥ 2</code>
Vật 2	3	4	<code>dp[2]=3, dp[3..4]=4</code>	<code>dp[5]=7 (1+2), dp[6..7]=7</code>	Chọn kết hợp 1 và 2 đạt 7
Vật 3	4	5	<code>dp[2]=3, dp[3]=4, dp[4]=5</code>	<code>dp[6]=8 (1+3), dp[7]=9 (2+3)</code>	Cập nhật cap 7 lên 9
Vật 4	5	6	Giữ nguyên	<code>dp[7] = max(9, dp[2]+6=9)</code>	*V_max = 9, W_used = 7*

19.4 Bài Tập Tiêu Chuẩn

Bài toán: Bài 19.1 - Các Bài Toán DP Kinh Điển: Balo 0/1 & Tối Ưu Hóa Giá Trị

(Nguồn: Nền tảng C++14 - Quy hoạch động kinh điển)

Cho N đồ vật ($1 \leq N \leq 15$) và một chiếc balo có sức chứa trọng lượng tối đa là W ($1 \leq W \leq 100$). Mỗi đồ vật i có trọng lượng w_i và giá trị v_i ($1 \leq w_i \leq 100, 1 \leq v_i \leq 1000$). Mỗi đồ vật chỉ được chọn nhiều nhất 1 lần (hoặc chọn, hoặc không chọn).

Em hãy tính toán:

- Tổng giá trị lớn nhất $V_{\max}$ có thể xếp vào balo mà tổng trọng lượng không vượt quá W .
- Tổng trọng lượng thực tế nhỏ nhất W_{used} của các đồ vật được chọn để đạt được đúng giá trị cực đại $V_{\max}$ đó.

Đầu vào (Input): Dòng 1 ghi hai số nguyên N, W . N dòng tiếp theo, mỗi dòng ghi hai số nguyên w_i, v_i .

Đầu ra (Output): In ra hai số nguyên `V_max W_used` cách nhau bởi khoảng trắng.

Ví dụ mẫu:

Đầu vào (Input):

4 7
2 3
3 4
4 5
5 6

Đầu ra (Output):

9 7

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	4 7 2 3 3 4 4 5 5 6	
2	Warm-up	3 4 1 2 2 3 3 4	
3	Warm-up	3 8 3 30 4 50 5 60	
4	Edge case	1 10 15 100	
5	Edge case	1 5 5 20	

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
6	Edge case	3 1 2 10 3 20 4 30	[.....]
7	Trick case	4 10 5 10 5 10 5 10 5 10	[.....]
8	Trick case	3 10 2 50 8 1 10 50	[.....]
9	Mental challenge	5 10 2 6 2 10 3 12 4 16 5 22	[.....]
10	Mental challenge	6 12 1 2 3 8 4 10 5 15 7 20 2 5	[.....]

Chương 20: Lý Thuyết Đồ Thị & Cách Biểu Diễn

[THUẬT NGỮ KHOA HỌC] Cấu Trúc Đồ Thị & Biểu Diễn Danh Sách Kề

Từ nguyên học: Lý thuyết đồ thị ra đời năm 1736 khi Leonhard Euler giải quyết bài toán nổi tiếng 7 cây cầu ở Königsberg, khởi đầu cho ngành topo học và mạng lưới rời rạc.

Mô hình trực giác: Đồ thị $G = (V, E)$ là mô hình trừu tượng hóa mối quan hệ giữa các thực thể: mỗi thực thể là một đỉnh (Vertex), và mỗi quan hệ/kết nối là một cạnh (Edge). Lựa chọn cấu trúc dữ liệu để biểu diễn đồ thị trong bộ nhớ máy tính quyết định trực tiếp đến tính khả thi về thời gian và không gian của mọi thuật toán tìm kiếm đường đi.

20.1 Các Khái Niệm Cơ Bản Trong Lý Thuyết Đồ Thị

- Đỉnh (V) và Cạnh (E):** Đồ thị gồm tập đỉnh V và tập cạnh E .
- Đồ thị có hướng vs vô hướng:** Cạnh có hướng là cặp có thứ tự (u, v) ; cạnh vô hướng là tập $\{u, v\}$.
- Bậc của đỉnh ($\deg(v)$):**
 - Trong đồ thị vô hướng: số lượng cạnh gắn vào đỉnh v .
 - Trong đồ thị có hướng: bán bậc vào $\deg^-(v)$ (số cung đi vào) và bán bậc ra $\deg^+(v)$ (số cung đi ra).
- Bổ đề bắt tay (Handshaking Lemma):** Tổng bậc của tất cả các đỉnh bằng hai lần số cạnh:

$$\sum_{v \in V} \deg(v) = 2 |E|$$

Hệ quả: Số lượng đỉnh có bậc lẻ luôn là một số chẵn!

20.2 So Sánh 3 Cấu Trúc Biểu Diễn Đồ Thị Trong C++14

1. Ma Trận Kề

- Mảng 2D `matrix[u][v]`.
- Bộ nhớ: $O(V^2)$.
- Kiểm tra $(u, v) \in E$: $O(1)$.
- Duyệt đỉnh kề: $O(V)$.
- Phù hợp:** Đồ thị dày ($E \approx V^2$), $V \leq 1000$.

2. Danh Sách Cạnh

- `vector<pair<int, int>>`.
- Bộ nhớ: $O(E)$.
- Kiểm tra (u, v) : $O(E)$.
- Duyệt đỉnh kề: $O(E)$.
- Phù hợp:** Thuật toán Kruskal (MST), Bellman-Ford.

3. Danh Sách Kề

- `vector<int> adj[N]`.
- Bộ nhớ: $O(V + E)$.
- Kiểm tra (u, v) : $O(\deg(u))$.
- Duyệt đỉnh kề: $O(\deg(u))$.
- Chuẩn mực CP:** BFS, DFS, Dijkstra, TopoSort.

[LƯU Ý QUAN TRỌNG] Cảnh báo kích thước bộ nhớ ma trận kề

Nếu $V = 10^5$, một ma trận kề kiểu `bool matrix[V][V]` đòi hỏi xấp xỉ 10^{10} bytes (≈ 10 GB RAM), lập tức bị lỗi Memory Limit Exceeded (MLE). Danh sách kề `vector<int> adj[V]` chỉ tiêu tốn $O(V + E)$, với $V, E \leq 2 \times 10^5$ chỉ mất khoảng 15 MB RAM.

20.3 Cài Đặt Danh Sách Kề Chuẩn Mực Trong C++14

[MỞ XẺ CÚ PHÁP C++14]	
<pre>std::vector<std::vector<int>> adj(n + 1); adj[u].push_back(v); adj[v].push_back(u);</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>std::vector<std::vector<int>> adj(n + 1);</code>	Khởi tạo vector chứa n+1 danh sách kề (đánh số 1-indexed an toàn).
<code>adj[u].push_back(v);</code>	Thêm đỉnh v vào danh sách lân cận của đỉnh u.
<code>adj[v].push_back(u);</code>	Với đồ thị vô hướng, cạnh đối xứng bắt buộc phải thêm ngược lại u vào adj[v].

20.4 Bảng Chạy Bàn Trên Giấy: Đồ Thị 4 Đỉnh Với Các Cạnh (1, 2), (2, 3), (3, 4)

Đỉnh v	Danh sách kề adj[v]	Bậc deg(v)	Phân loại đỉnh	Kiểm tra đơn đồ thị
1	[2]	1	Đỉnh treo (lá)	Không có khuyên
2	[1, 3]	2	Đỉnh trung gian	Không có cạnh lặp
3	[2, 4]	2	Đỉnh trung gian	Không có cạnh lặp
4	[3]	1	Đỉnh treo (lá)	Không có khuyên
Tổng kết	-	<code>max_deg = 2</code> (Đỉnh 2)	0 đỉnh cô lập	*Đơn đồ thị: 1 (Hợp lệ)*

20.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 20.1 - Biểu Diễn Đồ Thị & Phân Tích Bậc Đỉnh

(Nguồn: Nền tảng C++14 - Lý thuyết đồ thị cơ bản)

Cho một đồ thị vô hướng gồm N đỉnh (được đánh số từ 1 đến N) và M cạnh ($1 \leq N \leq 30, 0 \leq M \leq 100$). Các cạnh được cho dưới dạng danh sách cặp đỉnh (u_i, v_i) .

Em hãy xây dựng biểu diễn đồ thị bằng danh sách kề và tính toán:

- Bậc lớn nhất của một đỉnh trong đồ thị (`max_deg`).
- Đỉnh có bậc lớn nhất (`best_vertex`). Nếu có nhiều đỉnh cùng đạt bậc lớn nhất, chọn đỉnh có chỉ số nhỏ nhất.
- Số lượng đỉnh cô lập trong đồ thị (`isolated_count`), tức các đỉnh có bậc bằng 0.

4. Kiểm tra xem đồ thị có phải là đơn đồ thị (Simple Graph) hay không (`is_simple`): in ra 1 nếu là đơn đồ thị, 0 nếu chứa cạnh khuyên hoặc đa cạnh song song).

Đầu vào (Input): Dòng 1 ghi hai số nguyên N, M . M dòng tiếp theo, mỗi dòng ghi hai số nguyên u, v .

Đầu ra (Output): In ra 4 số nguyên `max_deg best_vertex isolated_count is_simple` cách nhau bởi khoảng trắng.

Ví dụ mẫu:

Đầu vào (Input):

4 3

1 2

2 3

3 4

Đầu ra (Output):

2 2 0 1

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì):

Đáp án đối chiếu xem tại Phụ lục cuối sách

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	4 3 1 2 2 3 3 4	[.....]
2	Warm-up	5 4 1 2 1 3 1 4 1 5	[.....]
3	Warm-up	3 3 1 2 2 3 3 1	[.....]
4	Edge case	1 0	[.....]
5	Edge case	4 0	[.....]
6	Edge case	2 1 1 1	[.....]
7	Trick case	3 2 1 2 1 2	[.....]
8	Trick case	5 4 1 2 2 1 3 3 4 5	[.....]
9	Mental challenge	6 5 1 2 2 3 4 5 5 6 4 6	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
10	Mental challenge	7 6 1 2 1 3 1 4 2 3 5 6 7 7	[.....]

Chương 21: Duyệt Đồ Thị Cơ Bản BFS & DFS

[THUẬT NGỮ KHOA HỌC] Duyệt Đồ Thị, Liên Thông & Đồ Thị Hai Phía

Từ nguyên học: Duyệt theo chiều sâu (DFS) và duyệt theo chiều rộng (BFS) được phát triển vào thế kỷ 19 để giải mê cung (thuật toán Trémaux) và chính thức hóa bởi Edward F. Moore (1959) trong bài toán tìm đường ngắn nhất.

Mô hình trực giác: Duyệt đồ thị là hành vi khám phá toàn bộ không gian đỉnh và cạnh một cách có hệ thống. DFS sử dụng cơ chế Ngăn xếp (Stack/Recursion) để đi sâu nhất có thể dọc theo một nhánh trước khi quay lui. BFS sử dụng Hàng đợi (Queue) để lan truyền đồng tâm theo từng tầng sóng, đảm bảo tìm ra đường đi có ít cạnh nhất trên đồ thị không trọng số.

21.1 So Sánh Bản Chất: DFS Đối Đầu BFS

1. DFS (Depth-First Search)

- Cấu trúc:** Call Stack hoặc `std::stack`.
- Cơ chế:** Thăm sâu đến tận cùng rồi Backtrack.
- Bộ nhớ:** $O(V)$ phụ thuộc vào độ sâu cây DFS.
- Ứng dụng:** Đếm thành phần liên thông, tìm chu trình, khớp/cầu, topo-sort, tô màu 2 phía.

2. BFS (Breadth-First Search)

- Cấu trúc:** Hàng đợi `std::queue`.
- Cơ chế:** Lan tỏa theo từng mức khoảng cách (Layer by Layer).
- Bộ nhớ:** $O(V)$ lưu trữ biên sóng duyệt.
- Ứng dụng:** Đường đi ngắn nhất không trọng số, tìm thành phần liên thông, đồ thị hai phía.

21.2 Tìm Thành Phần Liên Thông (Connected Components)

Trong đồ thị vô hướng, một thành phần liên thông là một đồ thị con tối đại mà giữa hai đỉnh bất kỳ luôn tồn tại đường đi.

- Thuật toán: Dùng mảng `visited[]` đánh dấu các đỉnh đã thăm.
- Duyệt qua tất cả các đỉnh $i = 1..N$. Nếu đỉnh i chưa được thăm (`!visited[i]`), ta khởi động một lượt duyệt BFS/DFS mới, tăng biến đếm số thành phần liên thông lên 1, và đếm tổng số đỉnh thuộc thành phần đó.

21.3 Đồ Thị Hai Phía (Bipartite Graph) & Tô 2 Màu

Một đồ thị là hai phía nếu tập đỉnh có thể chia thành hai tập rời rạc V_1, V_2 sao cho mọi cạnh chỉ nối giữa một đỉnh thuộc V_1 và một đỉnh thuộc V_2 .

Định lý König: Một đồ thị là đồ thị hai phía khi và chỉ khi không chứa chu trình có độ dài lẻ.

[MÔ XẼ CÚ PHÁP C++14]	
<pre>if (color[v] == -1) { color[v] = 1 - color[u]; q.push(v); } else if (color[v] == color[u]) { is_bipartite = false; }</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>if (color[v] == -1)</code>	Đỉnh v chưa được tô màu: gán màu đối nghịch với đỉnh u.
<code>color[v] = 1 - color[u];</code>	Công thức đảo màu tinh tế: 0 biến thành 1, 1 biến thành 0.
<code>else if (color[v] == color[u])</code>	Đỉnh kề đã được tô nhưng có cùng màu với đỉnh hiện tại.
<code>is_bipartite = false;</code>	Phát hiện xung đột màu: đồ thị chứa chu trình lẻ, không thể là đồ thị hai phía.

21.4 Bảng Chạy Bàn Trên Giấy: Đồ Thị 6 Đỉnh 2 Thành Phần

Đồ thị gồm các cạnh (1, 2), (2, 3), (3, 1) (thành phần 1) và (4, 5), (5, 6) (thành phần 2).

Bước	Đỉnh u	Hành động / Màu	Hàng đợi queue	Trạng thái Bipartite
1	1	Khởi tạo thành phần 1, color[1]=0	[1]	Khởi đầu hợp lệ
2	1	Pop 1 → duyệt 2 (color=1), 3 (color=1)	[2, 3]	Tô màu đối nghịch
3	2	Pop 2 → duyệt 3: color[3]=color[2]=1		*Xung đột màu: Chu trình lẻ C3!*
4	4	Khởi tạo thành phần 2, color[4]=0	[4]	is_bipartite = false
5	4	Pop 4 → duyệt 5 (color=1)	[5]	-
6	5	Pop 5 → duyệt 6 (color=0)	[6]	-
Kết thúc	-	2 thành phần, max_size = 3	rỗng	*is_bipartite = 0 (Không phải hai phía)*

21.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 21.1 - Duyệt Đồ Thị Cơ Bản: Thành Phần Liên Thông & Đồ Thị Hai Phía

(Nguồn: Nền tảng C++14 - Duyệt đồ thị cơ bản)

Cho một đồ thị vô hướng gồm N đỉnh (được đánh số từ 1 đến N) và M cạnh ($1 \leq N \leq 30, 0 \leq M \leq 100$). Đồ thị có thể không liên thông và có thể chứa nhiều thành phần độc lập.

Em hãy sử dụng thuật toán duyệt đồ thị (BFS hoặc DFS) để tính toán:

- Tổng số lượng thành phần liên thông của đồ thị (`comp_count`).
- Kích thước (số lượng đỉnh) của thành phần liên thông lớn nhất (`max_comp_size`).
- Kiểm tra xem toàn bộ đồ thị có phải là đồ thị hai phía (Bipartite Graph) hay không (`is_bipartite` : in ra 1 nếu là đồ thị hai phía, 0 nếu không).

Đầu vào (Input): Dòng 1 ghi hai số nguyên N, M . M dòng tiếp theo, mỗi dòng ghi hai số nguyên u, v .

Đầu ra (Output): In ra 3 số nguyên `comp_count max_comp_size is_bipartite` cách nhau bởi khoảng trắng.

Ví dụ mẫu:

Đầu vào (Input):

6 5

1 2

2 3

3 1

4 5

5 6

Đầu ra (Output):

2 3 0

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): <i>Đáp án đối chiếu xem tại Phụ lục cuối sách</i>			
Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	6 5 1 2 2 3 3 1 4 5 5 6	[.....]
2	Warm-up	4 3 1 2 2 3 3 4	[.....]
3	Warm-up	4 4 1 2 2 3 3 4 4 1	[.....]
4	Edge case	1 0	[.....]
5	Edge case	5 0	[.....]
6	Edge case	2 1 1 2	[.....]
7	Trick case	3 3 1 2 2 3 3 1	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
8	Trick case	5 5 1 2 2 3 3 4 4 5 5 1	[.....]
9	Mental challenge	7 6 1 2 2 3 3 4 5 6 6 7 5 7	[.....]
10	Mental challenge	8 6 1 2 3 4 5 6 7 8 1 3 5 7	[.....]

Chương 22: Đồ Thị Không Chu Trình (DAG) & Sắp Xếp Tô-pô

[THUẬT NGỮ KHOA HỌC] Đồ Thị DAG & Thủ Tục Tuyển Tính Hóa Tô-pô

Từ nguyên học: Sắp xếp tô-pô xuất phát từ phương pháp sơ đồ mạng PERT (Program Evaluation and Review Technique) trong công nghệ quản lý dự án những năm 1950, và thuật toán Arthur Kahn công bố năm 1962.

Mô hình trực giác: DAG (Directed Acyclic Graph) mô hình hóa mối quan hệ phụ thuộc nhân quả không hồi quy (ví dụ: môn học tiên quyết, quy trình biên dịch phần mềm). Sắp xếp tô-pô duỗi thẳng các đỉnh của DAG thành một hàng ngang sao cho mọi mũi tên đều chỉ từ trái sang phải. Thủ tục này mở ra cánh cửa cho Quy hoạch động trên đồ thị: giải quyết bài toán đường đi dài nhất hoặc đếm số cách đi mà không bao giờ sợ bị lặp vô tận.

22.1 Định Nghĩa DAG & Sắp Xếp Tô-pô (Topological Sort)

- DAG:** Đồ thị có hướng không chứa bất kỳ chu trình có hướng nào.
- Thủ tục tô-pô:** Một hoán vị các đỉnh $(v_1, v_2, \dots, v_N)$ sao cho với mọi cung có hướng $u \rightarrow v$, đỉnh u luôn xuất hiện trước đỉnh v trong dãy hoán vị ($\text{pos}(u) < \text{pos}(v)$).
- Đồ thị có thủ tục tô-pô khi và chỉ khi nó là một DAG.

22.2 Thuật Toán Kahn: Xóa Đỉnh Có Bán Bậc Vào Bằng 0

Thuật toán Kahn mô phỏng trực giác con người:

- Tính bán bậc vào `in_degree[v]` cho mọi đỉnh v .
- Đưa các đỉnh có `in_degree[v] == 0` vào hàng đợi (dùng `std::priority_queue` nếu cần thủ tục từ điển nhỏ nhất).
- Lặp lại: Lấy đỉnh u ra khỏi hàng đợi, đưa vào kết quả; với mỗi cung $u \rightarrow v$, giảm `in_degree[v]`. Nếu `in_degree[v] == 0`, đẩy v vào hàng đợi.
- Kiểm tra chu trình:** Nếu số đỉnh lấy ra nhỏ hơn N , đồ thị chứa chu trình có hướng!

[MÔ XẼ CÚ PHÁP C++14]

```
while (!pq.empty()) { int u = pq.top(); pq.pop(); for (int v : adj[u]) { if (--in_degree[v] == 0) pq.push(v); } }
```

Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>while (!pq.empty())</code>	Lặp chừng nào vẫn còn đỉnh không bị ràng buộc bởi bất kỳ đỉnh nào khác.
<code>int u = pq.top(); pq.pop();</code>	Trích xuất đỉnh có chỉ số nhỏ nhất (đảm bảo thứ tự từ điển tối ưu).
<code>for (int v : adj[u])</code>	Duyệt qua các đỉnh phụ thuộc nhận mũi tên từ u.
<code>if (--in_degree[v] == 0) pq.push(v);</code>	Triệt tiêu sự phụ thuộc u; nếu v không còn phụ thuộc nào, đẩy vào hàng đợi.

22.3 Quy Hoạch Động Trên DAG (DP on DAG)

Do mọi cung đều đi theo chiều tiến của thứ tự tô-pô, DAG không có chu trình nên ta có thể áp dụng Quy hoạch động trực tiếp theo thứ tự tô-pô. **Bài toán tìm đường đi dài nhất (Longest Path):**

$$dp[v] = \max_{(u,v) \in E} (dp[u] + 1)$$

Khởi tạo $dp[v] = 0$ cho mọi đỉnh. Độ dài đường đi dài nhất trong toàn bộ DAG là $\max_{v \in V} dp[v]$.

22.4 Bảng Chạy Bàn Trên Giấy: Đồ Thị 4 Đỉnh $(1 \rightarrow 2), (1 \rightarrow 3), (2 \rightarrow 4), (3 \rightarrow 4)$

Lượt	Đỉnh u lấy ra	Cập nhật bán bậc vào	Cập nhật $dp[v]$ (Đường đi)	Hàng đợi pq
Khởi tạo	-	in_deg: [-, 0, 1, 1, 2]	dp: [-, 0, 0, 0, 0]	[1]
1	1	<code>in_deg[2]=0, in_deg[3]=0</code>	<code>dp[2]=1, dp[3]=1</code>	[2, 3]
2	2	<code>in_deg[4] = 1</code>	<code>dp[4] = max(0, dp[2]+1=2) = 2</code>	[3]
3	3	<code>in_deg[4] = 0 (Đẩy 4 vào pq)</code>	<code>dp[4] = max(2, dp[3]+1=2) = 2</code>	[4]
4	4	Không còn đỉnh phụ thuộc	-	rỗng

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
3	Warm-up	4 3 4 3 3 2 2 1	[.....]
4	Edge case	1 0	[.....]
5	Edge case	3 0	[.....]
6	Edge case	2 1 1 2	[.....]
7	Trick case	3 3 1 2 2 3 3 1	[.....]
8	Trick case	4 4 1 2 2 3 3 2 3 4	[.....]
9	Mental challenge	5 5 1 4 2 4 3 5 4 5 1 2	[.....]
10	Mental challenge	6 7 1 2 1 3 2 4 3 4 4 5 4 6 5 6	[.....]

Chương 23: Cây (Trees) & Các Thuộc Tính Đặc Biệt

[THUẬT NGỮ KHOA HỌC] Cấu Trúc Cây & Thuật Toán 2 Lần DFS Tìm Đường Kính

Từ nguyên học: Khái niệm cây được Arthur Cayley đưa ra năm 1857 khi nghiên cứu các đồng phân hóa học hữu cơ phân nhánh, trở thành cấu trúc dữ liệu nền tảng nhất của khoa học máy tính.

Mô hình trực giác: Một cái cây là một đồ thị liên thông nhưng tối giản đến mức thanh khiết: nó có N đỉnh và đúng $N - 1$ cạnh, không có chu trình. Giữa hai đỉnh bất kỳ trên cây chỉ tồn tại duy nhất một đường đi đơn. Mọi nhánh cây khi cắt bỏ một cạnh lập tức bị chia tách thành hai thành phần độc lập, mở ra các tính chất quy nạp tuyệt đẹp về kích thước cây con và đường kính cực đại.

23.1 Định Nghĩa & Các Thuộc Tính Toán Học Của Cây

Cho đồ thị vô hướng $G = (V, E)$ có N đỉnh. Các mệnh đề sau tương đương:

- G là một cây (liên thông và không có chu trình).
- G liên thông và có đúng $N - 1$ cạnh.
- G không có chu trình và có đúng $N - 1$ cạnh.
- Giữa hai đỉnh bất kỳ trên cây tồn tại duy nhất một đường đi đơn.

23.2 Cây Có Gốc (Rooted Tree) & Kích Thuộc Cây Con

Khi chọn đỉnh 1 làm gốc (Root):

- Quan hệ cha - con: Mọi đỉnh $u \neq 1$ có duy nhất một nút cha $p = \text{parent}[u]$.
- Chiều sâu (Depth): $\text{depth}[1] = 0$, $\text{depth}[v] = \text{depth}[u] + 1$ với v là con của u .
- Nút lá (Leaves): Nút không có con nào (bậc bằng 1 khi khác gốc).
- Kích thước cây con (Subtree Size):

$$\text{subtree_size}[u] = 1 + \sum_{v \in \text{children}(u)} \text{subtree_size}[v]$$

23.3 Đường Kính Của Cây (Tree Diameter) & Thuật Toán 2 Lần DFS

Đường kính cây D là khoảng cách lớn nhất giữa hai đỉnh bất kỳ trên cây:

$$D = \max_{u, v \in V} \text{dist}(u, v)$$

Thuật toán 2 lần DFS (Double DFS Algorithm):

- DFS lần 1:** Xuất phát từ một đỉnh bất kỳ (thường là đỉnh 1), tìm đỉnh A có khoảng cách xa nhất.
- DFS lần 2:** Xuất phát từ đỉnh A , tìm đỉnh B có khoảng cách xa nhất.

Kết luận: Khoảng cách $\text{dist}(A, B)$ chính là đường kính của cây!

[TIP] Chứng minh tính đúng đắn trên giấy của 2 lần DFS

Giả sử đường kính thực sự của cây nối giữa hai đỉnh U và V . Khi ta xuất phát từ đỉnh 1 bất kỳ, đỉnh A xa nhất tìm được **chắc chắn phải là một trong hai đầu mút U hoặc V** (bởi nếu không, đường đi từ 1 đến A kết hợp với đường kính sẽ tạo ra một đường đi đơn dài hơn $\text{dist}(U, V)$, mâu thuẫn với giả thiết $\text{dist}(U, V)$ là đường kính lớn nhất).

[MÔ XẼ CÚ PHÁP C++14]	
<pre>void dfs(int u, int p, int d) { if (d > max_dist) { max_dist = d; farthest = u; } for (int v : adj[u]) if (v != p) dfs(v, u, d + 1); }</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>void dfs(int u, int p, int d)</code>	Hàm DFS nhận đỉnh hiện tại u , đỉnh cha p để chống đi ngược, và khoảng cách d .
<code>if (d > max_dist)</code>	Kiểm tra và cập nhật đỉnh xa nhất từng ghi nhận.
<code>for (int v : adj[u])</code>	Duyệt qua mọi đỉnh kề của u .
<code>if (v != p)</code>	Ngăn đệ quy quay lại đỉnh cha, bảo toàn tính chất cây không chu trình.

23.4 Bảng Chạy Bàn Trên Giấy: Cây 5 Đỉnh Gốc 1 Với Cạnh $(1, 2), (1, 3), (2, 4), (2, 5)$

Lần DFS	Đỉnh xuất phát	Đỉnh xa nhất tìm được	Khoảng cách max	Ý nghĩa hình học
DFS 1	1	4 (hoặc 5)	2	Tìm được một đầu mút của đường kính: $A = 4$
DFS 2	4	3 (qua $4 \rightarrow 2 \rightarrow 1 \rightarrow 3$)	3	*Tìm được đầu mút đối diện: $B = 3$ *
Gốc 1	Duyệt cây con	<code>subtree_size[2] = 3</code>	<code>subtree_size[3] = 1</code>	<code>max_child_subtree = 3</code>
Tổng kết	-	<code>max_depth = 2</code> (tới 4, 5)	Số lá: 3 (3, 4, 5)	*Đường kính $D = 3$ *

23.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 23.1 - Cây & Thuộc Tính Cốt Lõi: Chiều Cao, Đường Kính & Cây Con

(Nguồn: Nền tảng C++14 - Lý thuyết cây & DFS)

Cho một cây gồm N đỉnh ($1 \leq N \leq 30$) và $N - 1$ cạnh, các đỉnh được đánh số từ 1 đến N . Cây được coi là có gốc tại đỉnh 1.

Em hãy tính toán:

- 1. Đường kính của cây (`diameter`): Khoảng cách lớn nhất giữa hai đỉnh bất kỳ trên cây (sử dụng thuật toán 2 lần DFS).
- 2. Chiều cao lớn nhất của cây (`max_depth`): Khoảng cách lớn nhất từ đỉnh gốc 1 đến một đỉnh bất kỳ trên cây ($\text{depth}[1] = 0$).
- 3. Số lượng nút lá của cây (`leaf_count`): Số lượng nút trong cây không có con nào khi coi 1 là gốc.
- 4. Kích thước cây con lớn nhất trong số các cây con của các con trực tiếp của gốc 1 (`max_child_subtree_size`). Nếu $N = 1$, giá trị này là 0.

Đầu vào (Input): Dòng 1 ghi số nguyên N . $N - 1$ dòng tiếp theo, mỗi dòng ghi hai số nguyên u, v mô tả một cạnh.

Đầu ra (Output): In ra 4 số nguyên `diameter max_depth leaf_count max_child_subtree_size` cách nhau bởi khoảng trắng.

Ví dụ mẫu:

Đầu vào (Input):

```
5
1 2
1 3
2 4
2 5
```

Đầu ra (Output):

```
3 2 3 3
```

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	5 1 2 1 3 2 4 2 5	[.....]
2	Warm-up	4 1 2 2 3 3 4	[.....]
3	Warm-up	5 1 2 1 3 1 4 1 5	[.....]
4	Edge case	1	[.....]
5	Edge case	2 1 2	[.....]
6	Edge case	3 1 2 1 3	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
7	Trick case	6 1 2 2 3 3 4 4 5 5 6	[.....]
8	Trick case	6 2 1 2 3 2 4 2 5 2 6	[.....]
9	Mental challenge	7 1 2 1 3 2 4 2 5 3 6 3 7	[.....]
10	Mental challenge	8 1 2 2 3 3 4 1 5 5 6 6 7 7 8	[.....]

Chương 24: Đường Đi Ngắn Nhất Với Thuật Toán Dijkstra

[THUẬT NGỮ KHOA HỌC] Thuật Toán Dijkstra & Hàng Đợi Ưu Tiên Min-Heap

Từ nguyên học: Được nhà khoa học máy tính người Hà Lan Edsger W. Dijkstra phát minh năm 1956 chỉ trong 20 phút khi uống cà phê cùng vị hôn thê, và công bố năm 1959.

Mô hình trực giác: Dijkstra là sự kết hợp hoàn hảo giữa Thuật toán Tham lam (Greedy) và Quy hoạch động. Xuất phát từ đỉnh nguồn, tại mỗi bước thuật toán luôn 'chốt hạ' khoảng cách ngắn nhất của đỉnh chưa thăm có nhãn khoảng cách nhỏ nhất. Sau đó, nó dùng đỉnh này để thư giãn (Relaxation) khoảng cách tới các đỉnh kề. Bằng cách sử dụng Min-Heap (priority_queue), thời gian trích xuất đỉnh tối ưu giảm từ $O(V)$ xuống $O(\log V)$.

24.1 Bài Toán Đường Đi Ngắn Nhất Nguồn Đơn (SSSP)

Cho đồ thị $G = (V, E)$ có trọng số cạnh không âm $w(e) \geq 0$. Cần tìm độ dài đường đi ngắn nhất từ một đỉnh nguồn S tới tất cả các đỉnh còn lại trong đồ thị.

[DANGER] Điều kiện tiên quyết: Trọng số không âm

Thuật toán Dijkstra **hoàn toàn thất bại** nếu đồ thị xuất hiện cạnh có trọng số âm ($w(e) < 0$). **Phản ví dụ:** Xét đồ thị có đỉnh $S \rightarrow A$ (chi phí 2), $S \rightarrow B$ (chi phí 5), và $B \rightarrow A$ (chi phí -4). Dijkstra sẽ chốt hạ $\text{dist}[A] = 2$ ngay từ đầu, trong khi đường đi thực tế $S \rightarrow B \rightarrow A$ có chi phí $5 + (-4) = 1 < 2$. Để xử lý cạnh âm, bắt buộc phải dùng thuật toán Bellman-Ford hoặc SPFA.

24.2 Cài Đặt Chuẩn C++14 Với `std::priority_queue`

Cú pháp khai báo Min-Heap trong C++14:

```
std::priority_queue<std::pair<long long, int>,  
                  std::vector<std::pair<long long, int>>,  
                  std::greater<std::pair<long long, int>>> pq;
```

Heap lưu trữ các cặp (khoảng cách d, đỉnh u). Toán tử `std::greater` đảm bảo phần tử có khoảng cách nhỏ nhất luôn nằm ở đỉnh heap (`pq.top()`).

[MỖ XẼ CÚ PHÁP C++14]	
<pre>if (d > dist[u]) continue; for (auto& e : adj[u]) { if (dist[u] + e.w < dist[e.to]) { dist[e.to] = dist[u] + e.w; pq.push({dist[e.to], e.to}); } }</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>if (d > dist[u]) continue;</code>	Kỹ thuật Lazy Deletion: Bỏ qua các bản ghi lỗi thời đã có đường đi tốt hơn được xử lý trước đó.
<code>for (auto& e : adj[u])</code>	Duyệt qua các cạnh đi ra từ đỉnh u vừa được chốt hạ.
<code>dist[u] + e.w < dist[e.to]</code>	Phép thử thư giãn (Edge Relaxation): Kiểm tra đi qua u có tốt hơn không.
<code>pq.push({dist[e.to], e.to});</code>	Đẩy nhãn khoảng cách mới vào heap để tiếp tục lan tỏa sóng.

24.3 Bảng Chạy Bàn Trên Giấy: Đồ Thị 4 Đỉnh Với $S = 1$ Đến $T = 4$

Các cạnh: $(1 \rightarrow 2, 2)$, $(1 \rightarrow 3, 5)$, $(2 \rightarrow 4, 4)$, $(3 \rightarrow 4, 1)$.

Bước	Pop (d, u)	Mảng dist[]	Thu giản các cạnh	Nội dung Min-Heap
Khởi tạo	-	[0, INF, INF, INF]	<code>dist[1] = 0</code>	[(0, 1)]
1	(0, 1)	[0, 2, 5, INF]	<code>1→2: dist[2]=2;</code> <code>1→3: dist[3]=5</code>	[(2, 2), (5, 3)]
2	(2, 2)	[0, 2, 5, 6]	<code>2→4: dist[4] = 2 + 4 = 6</code>	[(5, 3), (6, 4)]
3	(5, 3)	[0, 2, 5, 6]	<code>3→4: 5 + 1 = 6</code> (Không nhỏ hơn 6)	[(6, 4)]
4	(6, 4)	[0, 2, 5, 6]	Đích đến 4 được chốt hạ	rỗng
Kết thúc	-	<code>*dist[4] = 6*</code>	<code>*Truy vết parent[4] = 2, parent[2] = 1*</code>	*Đường đi: 1 -> 2 -> 4*

24.4 Bài Tập Tiêu Chuẩn

Bài toán: Bài 24.1 - Thuật Toán Dijkstra Chuẩn Mục & Phục Hồi Đường Đi

(Nguồn: Nền tảng C++14 - Đường đi ngắn nhất Dijkstra)

Cho một đồ thị có hướng gồm N đỉnh (được đánh số từ 1 đến N) và M cung có trọng số không âm ($1 \leq N \leq 30, 1 \leq M \leq 100$). Em hãy tìm đường đi ngắn nhất từ đỉnh nguồn $S = 1$ đến đỉnh đích $T = N$.

Nếu không tồn tại đường đi từ 1 đến N , in ra **-1**. Nếu tồn tại đường đi:

1. In ra độ dài đường đi ngắn nhất `dist[N]`.
2. In ra số lượng cạnh trên đường đi ngắn nhất đó `edge_count`.
3. In ra đỉnh trung gian nằm ngay trước đỉnh N trên đường đi ngắn nhất (`parent[N]`).

Đầu vào (Input): Dòng 1 ghi hai số nguyên N, M . M dòng tiếp theo, mỗi dòng ghi ba số nguyên u, v, w mô tả cung từ u tới v có trọng số w .

Đầu ra (Output): Nếu không có đường đi, in ra `-1`. Nếu có đường đi, in ra 3 số nguyên `dist[N]` `edge_count` `parent[N]` cách nhau bởi khoảng trắng.

Ví dụ mẫu:

Đầu vào (Input):

4	4	
1	2	2
1	3	5
2	4	4
3	4	1

Đầu ra (Output):

6 2 2

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	4 4 1 2 2 1 3 5 2 4 4 3 4 1	[.....]
2	Warm-up	3 3 1 2 10 2 3 5 1 3 20	[.....]
3	Warm-up	4 3 1 2 3 2 3 4 3 4 5	[.....]
4	Edge case	1 0	[.....]
5	Edge case	3 1 1 2 5	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
6	Edge case	2 1 1 2 0	[.....]
7	Trick case	3 2 2 3 4 3 1 2	[.....]
8	Trick case	4 4 1 2 10 1 3 1 3 2 1 2 4 1	[.....]
9	Mental challenge	5 6 1 2 4 1 3 2 3 2 1 2 4 5 3 5 8 4 5 1	[.....]
10	Mental challenge	6 8 1 2 3 1 3 5 2 4 2 3 4 1 2 5 6 4 5 2 4 6 7 5 6 1	[.....]

Bài Tập Tích Hợp Tập 3: Đồ Thị, Dijkstra & Quy Hoạch Động Trên DAG

[THUẬT NGỮ KHOA HỌC] Hợp Nhất Đồ Thị & Quy Hoạch Động: Shortest Path DAG

Từ nguyên học: Mô hình quy hoạch động trên đồ thị đường đi ngắn nhất là dạng bài toán thượng thừa thường xuất hiện trong các kỳ thi VOI, ICPC Regional, và các nền tảng thi đấu như CSES/Codeforces.

Mô hình trực giác: Khi giải bài toán 'Đếm số lượng đường đi ngắn nhất' hoặc 'Tìm đường đi ngắn nhất có số cạnh ít nhất/nhiều nhất', việc chạy thuật toán Dijkstra đơn thuần chỉ cho ta một đường đi duy nhất. Bằng cách giữ lại toàn bộ các cạnh thỏa mãn điều kiện $\text{dist}[u] + w == \text{dist}[v]$, ta thu được một Đồ thị con không chu trình (Shortest Path DAG). Trên DAG này, ta áp dụng trọn vẹn sức mạnh của Quy hoạch động theo thứ tự khoảng cách tăng dần để đếm và tối ưu hóa mọi thuộc tính của mạng lưới.

1. Kiến Trúc Giải Pháp 2 Giai Đoạn

Quy trình giải quyết gồm 2 pha độc lập và chuẩn mực:

Giai Đoạn 1: Dijkstra Nguồn Đơn

- Chạy Dijkstra từ đỉnh nguồn 1 với Min-Heap `std::priority_queue`.
- Tính toán mảng khoảng cách tối ưu $\text{dist}[u]$ tới mọi đỉnh $u \in [1, N]$.
- Xác định tập các đỉnh có thể tiếp cận ($\text{dist}[u] \neq \text{INF}$).

Giai Đoạn 2: DP Trên Shortest Path DAG

- Do trọng số cạnh $w \geq 1$, thứ tự tăng dần của $\text{dist}[u]$ chính là một **thứ tự tô-pô** tự nhiên!
- Sắp xếp các đỉnh tiếp cận theo $\text{dist}[u]$.
- Duyệt từng đỉnh u , chuyển trạng thái cho các cạnh tối ưu: $\text{dist}[u] + w == \text{dist}[v]$.

2. Công Thức Chuyển Trạng Thái Quy Hoạch Động

Với mỗi cung $u \rightarrow v$ có trọng số w thỏa mãn $\text{dist}[u] + w == \text{dist}[v]$:

1. Đếm số đường đi ngắn nhất:

$$\text{ways}[v] = (\text{ways}[v] + \text{ways}[u]) \bmod (10^9 + 7)$$

2. Số cạnh ít nhất:

$$\text{min_edges}[v] = \min(\text{min_edges}[v], \text{min_edges}[u] + 1)$$

3. Số cạnh nhiều nhất:

$$\text{max_edges}[v] = \max(\text{max_edges}[v], \text{max_edges}[u] + 1)$$

[MÔ XẼ CÚ PHÁP C++14]	
<pre>if (dist[u] + w == dist[v]) { ways[v] = (ways[v] + ways[u]) % MOD; min_e[v] = min(min_e[v], min_e[u] + 1); max_e[v] = max(max_e[v], max_e[u] + 1); }</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>dist[u] + w == dist[v]</code>	Điều kiện tiên quyết xác nhận cạnh $u \rightarrow v$ thuộc Shortest Path DAG.
<code>ways[v] = (ways[v] + ways[u]) % MOD;</code>	Cộng dồn số lượng phương án đến từ u vào v .
<code>min_e[v] = min(... , min_e[u] + 1);</code>	Cập nhật lộ trình có ít nút trung gian nhất.
<code>max_e[v] = max(... , max_e[u] + 1);</code>	Cập nhật lộ trình ghé thăm nhiều trạm nhất.

3. Bảng Chạy Bàn Trên Giấy: Mạng Lưới 4 Đỉnh, 5 Cạnh

Các cạnh: $(1 \rightarrow 2, 2), (1 \rightarrow 3, 3), (2 \rightarrow 4, 3), (3 \rightarrow 4, 2), (1 \rightarrow 4, 5)$.

Đỉnh u	$dist[u]$	$ways[u]$	$min_edges[u]$	$max_edges[u]$	Ghi chú lan truyền
1	0	1	0	0	Khởi tạo gốc nguồn
2	2	1	1	1	Nhận từ 1 qua cạnh $(1 \rightarrow 2)$ $w=2$
3	3	1	1	1	Nhận từ 1 qua cạnh $(1 \rightarrow 3)$ $w=3$
4	5	3	1	2	*Nhận từ 1 ($e=1$), từ 2 ($e=2$), từ 3 ($e=2$)*
Kết quả	<code>min_dist = 5</code>	<code>ways = 3</code>	<code>min_edges = 1</code>	<code>max_edges = 2</code>	*Đáp án: 5 3 1 2*

4. Đề Bài Bài Tập Tích Hợp

Bài toán: Bài Tập Tích Hợp 3 - Lộ Trình Tối Ưu: Dijkstra & Quy Hoạch Động Trên Shortest Path DAG (Nguồn: Tổng hợp Tập 3 - Đồ thị nâng cao & DP)

Cho mạng lưới giao thông gồm N nút giao ($1 \leq N \leq 30$) và M tuyến đường một chiều ($1 \leq M \leq 100$), mỗi tuyến đường từ u đến v có độ dài nguyên dương w ($1 \leq w \leq 1000$). Em hãy khảo sát toàn diện hành trình di chuyển từ nút 1 đến nút N :

- Khoảng cách ngắn nhất từ nút 1 đến nút N (`min_dist`).
- Số lượng đường đi ngắn nhất khác nhau từ nút 1 đến nút N (`ways_count`, chia lấy dư cho $10^9 + 7$).
- Số tuyến đường (số cạnh) ít nhất của một đường đi ngắn nhất từ nút 1 đến nút N (`min_edges`).
- Số tuyến đường (số cạnh) nhiều nhất của một đường đi ngắn nhất từ nút 1 đến nút N (`max_edges`).

Đầu vào (Input): Dòng 1 ghi hai số nguyên N, M . M dòng tiếp theo, mỗi dòng ghi ba số nguyên u, v, w .

Đầu ra (Output): Nếu không có đường đi từ 1 đến N , in ra `-1`. Nếu có đường đi, in ra 4 số nguyên `min_dist ways_count min_edges max_edges` cách nhau bởi khoảng trắng.

Ví dụ mẫu:

Đầu vào (Input):

4 5

1 2 2

1 3 3

2 4 3

3 4 2

1 4 5

Đầu ra (Output):

5 3 1 2

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): <i>Đáp án đối chiếu xem tại Phụ lục cuối sách</i>			
Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	4 5 1 2 2 1 3 3 2 4 3 3 4 2 1 4 5	[.....]
2	Warm-up	3 3 1 2 4 2 3 6 1 3 10	[.....]
3	Warm-up	4 3 1 2 1 2 3 2 3 4 3	[.....]
4	Edge case	1 0	[.....]
5	Edge case	3 1 1 2 5	[.....]
6	Edge case	2 1 1 2 100	[.....]
7	Trick case	4 4 1 2 5 2 3 5 3 4 5 1 4 20	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
8	Trick case	3 2 2 3 1 3 1 1	[.....]
9	Mental challenge	5 7 1 2 1 1 3 1 2 4 2 3 4 2 2 5 5 3 5 5 4 5 2	[.....]
10	Mental challenge	6 9 1 2 2 1 3 2 2 4 3 3 4 3 4 5 1 2 5 4 3 5 4 5 6 2 4 6 3	[.....]

Phụ Lục: Đáp Án & Bảng Vết Thực Thi 10 Test Bàn Tay

[LÝ THUYẾT] **Hướng Dẫn Đối Chiếu Lời Giải Bằng Bút Chì**

Phụ lục này cung cấp toàn bộ đáp án chính thức và bảng vết thực thi (Execution Trace) từng bước cho tất cả 10 bài test của từng chương trong Tập 3. Sau khi đã tự giải và điền kết quả vào các ô trống bằng bút chì, các em hãy đối chiếu với bảng đáp án dưới đây để phát hiện chính xác bước tư duy nào bị nhầm lẫn.

Lời Giải Chi Tiết Chương 18: Bản Chất Quy Hoạch Động

Lời giải & Đáp án: Bài 18.1 - Bản Chất Quy Hoạch Động: Bậc Thang Chi Phí Tối Thiểu & Đếm Phương Án			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	4 10 15 20 10	25 1	Dãy 4 bậc thang cơ bản
2	5 1 2 1 1 3	5 1	Dãy 5 bậc thang xuất hiện trường hợp nhánh hòa điểm
3	3 5 10 20	25 1	Dãy 3 bậc thang tối ưu nhảy từ bậc 1
4	1 50	50 1	Cận dưới N=1 chỉ có đúng 1 bậc
5	2 100 200	200 1	Cận biên N=2 kết thúc ngay tại bậc 2
6	2 500 500	500 1	Cận biên N=2 với chi phí hai bậc bằng nhau
7	6 2 2 2 2 2 2	6 1	Mảng chi phí đều nhau toàn 2
8	6 1 1 1 1 1 1	3 1	Mảng chi phí tối thiểu toàn 1
9	7 3 2 4 1 5 2 6	11 1	Thử thách tính nhẩm 7 bậc xen kẽ chẵn lẻ
10	6 3 3 1 1 3 3	7 1	Thử thách tính nhẩm phân nhánh cân bằng

Bảng Vết Thực Thi Từng Bước (Chương 18)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	4
-	c	(0, 10, 15, 20, 10)
2	Initialize base cases dp[1] and dp[2]	-

Bước	Mô tả / Biến	Giá trị
-	dp[1]	10
-	ways[1]	1
-	dp[2]	15
-	ways[2]	1
13	Transition for step i	-
-	i	3
-	c[i]	20
-	dp[i-1]	15
-	dp[i-2]	10
-	dp[i]	30
-	ways[i]	1
14	Transition for step i	-
-	i	4
-	c[i]	10
-	dp[i-1]	30
-	dp[i-2]	15
-	dp[i]	25
-	ways[i]	1
99	Optimal answer reached	-
-	min_cost	25
-	ways	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	5
-	c	(0, 1, 2, 1, 1, 3)
2	Initialize base cases dp[1] and dp[2]	-
-	dp[1]	1
-	ways[1]	1
-	dp[2]	2
-	ways[2]	1
13	Transition for step i	-
-	i	3
-	c[i]	1
-	dp[i-1]	2
-	dp[i-2]	1
-	dp[i]	2
-	ways[i]	1
14	Transition for step i	-
-	i	4
-	c[i]	1

Bước	Mô tả / Biến	Giá trị
-	dp[i-1]	2
-	dp[i-2]	2
-	dp[i]	3
-	ways[i]	2
15	Transition for step i	-
-	i	5
-	c[i]	3
-	dp[i-1]	3
-	dp[i-2]	2
-	dp[i]	5
-	ways[i]	1
99	Optimal answer reached	-
-	min_cost	5
-	ways	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	3
-	c	(0, 5, 10, 20)
2	Initialize base cases dp[1] and dp[2]	-
-	dp[1]	5
-	ways[1]	1
-	dp[2]	10
-	ways[2]	1
13	Transition for step i	-
-	i	3
-	c[i]	20
-	dp[i-1]	10
-	dp[i-2]	5
-	dp[i]	25
-	ways[i]	1
99	Optimal answer reached	-
-	min_cost	25
-	ways	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	1
-	c	(0, 50)
2	Base case n = 1	-
-	min_cost	50

Bước	Mô tả / Biến	Giá trị
-	ways	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	2
-	c	(0, 100, 200)
2	Initialize base cases dp[1] and dp[2]	-
-	dp[1]	100
-	ways[1]	1
-	dp[2]	200
-	ways[2]	1
99	Optimal answer reached	-
-	min_cost	200
-	ways	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	2
-	c	(0, 500, 500)
2	Initialize base cases dp[1] and dp[2]	-
-	dp[1]	500
-	ways[1]	1
-	dp[2]	500
-	ways[2]	1
99	Optimal answer reached	-
-	min_cost	500
-	ways	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	6
-	c	(0, 2, 2, 2, 2, 2, 2)
2	Initialize base cases dp[1] and dp[2]	-
-	dp[1]	2
-	ways[1]	1
-	dp[2]	2
-	ways[2]	1
13	Transition for step i	-
-	i	3
-	c[i]	2
-	dp[i-1]	2
-	dp[i-2]	2

Bước	Mô tả / Biến	Giá trị
-	dp[i]	4
-	ways[i]	2
14	Transition for step i	-
-	i	4
-	c[i]	2
-	dp[i-1]	4
-	dp[i-2]	2
-	dp[i]	4
-	ways[i]	1
15	Transition for step i	-
-	i	5
-	c[i]	2
-	dp[i-1]	4
-	dp[i-2]	4
-	dp[i]	6
-	ways[i]	3
16	Transition for step i	-
-	i	6
-	c[i]	2
-	dp[i-1]	6
-	dp[i-2]	4
-	dp[i]	6
-	ways[i]	1
99	Optimal answer reached	-
-	min_cost	6
-	ways	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	6
-	c	(0, 1, 1, 1, 1, 1, 1)
2	Initialize base cases dp[1] and dp[2]	-
-	dp[1]	1
-	ways[1]	1
-	dp[2]	1
-	ways[2]	1
13	Transition for step i	-
-	i	3
-	c[i]	1
-	dp[i-1]	1
-	dp[i-2]	1

Bước	Mô tả / Biến	Giá trị
-	dp[i]	2
-	ways[i]	2
14	Transition for step i	-
-	i	4
-	c[i]	1
-	dp[i-1]	2
-	dp[i-2]	1
-	dp[i]	2
-	ways[i]	1
15	Transition for step i	-
-	i	5
-	c[i]	1
-	dp[i-1]	2
-	dp[i-2]	2
-	dp[i]	3
-	ways[i]	3
16	Transition for step i	-
-	i	6
-	c[i]	1
-	dp[i-1]	3
-	dp[i-2]	2
-	dp[i]	3
-	ways[i]	1
99	Optimal answer reached	-
-	min_cost	3
-	ways	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	7
-	c	(0, 3, 2, 4, 1, 5, 2, 6)
2	Initialize base cases dp[1] and dp[2]	-
-	dp[1]	3
-	ways[1]	1
-	dp[2]	2
-	ways[2]	1
13	Transition for step i	-
-	i	3
-	c[i]	4
-	dp[i-1]	2
-	dp[i-2]	3

Bước	Mô tả / Biến	Giá trị
-	dp[i]	6
-	ways[i]	1
14	Transition for step i	-
-	i	4
-	c[i]	1
-	dp[i-1]	6
-	dp[i-2]	2
-	dp[i]	3
-	ways[i]	1
15	Transition for step i	-
-	i	5
-	c[i]	5
-	dp[i-1]	3
-	dp[i-2]	6
-	dp[i]	8
-	ways[i]	1
16	Transition for step i	-
-	i	6
-	c[i]	2
-	dp[i-1]	8
-	dp[i-2]	3
-	dp[i]	5
-	ways[i]	1
17	Transition for step i	-
-	i	7
-	c[i]	6
-	dp[i-1]	5
-	dp[i-2]	8
-	dp[i]	11
-	ways[i]	1
99	Optimal answer reached	-
-	min_cost	11
-	ways	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n and cost array c	-
-	n	6
-	c	(0, 3, 3, 1, 1, 3, 3)
2	Initialize base cases dp[1] and dp[2]	-
-	dp[1]	3
-	ways[1]	1

Bước	Mô tả / Biến	Giá trị
-	dp[2]	3
-	ways[2]	1
13	Transition for step i	-
-	i	3
-	c[i]	1
-	dp[i-1]	3
-	dp[i-2]	3
-	dp[i]	4
-	ways[i]	2
14	Transition for step i	-
-	i	4
-	c[i]	1
-	dp[i-1]	4
-	dp[i-2]	3
-	dp[i]	4
-	ways[i]	1
15	Transition for step i	-
-	i	5
-	c[i]	3
-	dp[i-1]	4
-	dp[i-2]	4
-	dp[i]	7
-	ways[i]	3
16	Transition for step i	-
-	i	6
-	c[i]	3
-	dp[i-1]	7
-	dp[i-2]	4
-	dp[i]	7
-	ways[i]	1
99	Optimal answer reached	-
-	min_cost	7
-	ways	1

Lời Giải Chi Tiết Chương 19: Các Bài Toán DP Kinh Điển

Lời giải & Đáp án: Bài 19.1 - Các Bài Toán DP Kinh Điển: Balo 0/1 & Tối Ưu Hóa Giá Trị			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	4 7 2 3 3 4 4 5 5 6	9 7	Balo mẫu 4 đồ vật sức chứa 7
2	3 4 1 2 2 3 3 4	6 4	Balo 3 vật chọn cặp tối ưu đạt giá trị 6
3	3 8 3 30 4 50 5 60	90 8	Balo 3 vật giá trị cao đạt 90 tại trọng lượng 8
4	1 10 15 100	0 0	Đồ vật vượt quá sức chứa balo
5	1 5 5 20	20 5	Đồ vật duy nhất vừa vận sức chứa
6	3 1 2 10 3 20 4 30	0 0	Sức chứa tối thiểu W=1 nhỏ hơn mọi đồ vật
7	4 10 5 10 5 10 5 10 5 10	20 10	Các đồ vật có cùng trọng lượng và giá trị
8	3 10 2 50 8 1 10 50	51 10	Vật nhẹ giá trị cao kết hợp vật nặng giá trị thấp
9	5 10 2 6 2 10 3 12 4 16 5 22	44 10	Thử thách tính nhẩm balo 5 vật sức chứa 10
10	6 12 1 2 3 8 4 10 5 15 7 20 2 5	35 12	Thử thách tính nhẩm balo 6 vật sức chứa 12

Bảng Vết Thực Thi Từng Bước (Chương 19)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	4
-	W	7
11	Process item i	-
-	item_idx	1
-	weight	2
-	value	3
-	dp_row	(0, 0, 3, 3, 3, 3, 3, 3)
12	Process item i	-
-	item_idx	2
-	weight	3
-	value	4
-	dp_row	(0, 0, 3, 4, 4, 7, 7, 7)
13	Process item i	-
-	item_idx	3
-	weight	4
-	value	5
-	dp_row	(0, 0, 3, 4, 5, 7, 8, 9)
14	Process item i	-
-	item_idx	4
-	weight	5
-	value	6
-	dp_row	(0, 0, 3, 4, 5, 7, 8, 9)
99	Found optimal Knapsack solution	-
-	max_value	9
-	weight_used	7

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	3
-	W	4
11	Process item i	-
-	item_idx	1
-	weight	1
-	value	2
-	dp_row	(0, 2, 2, 2, 2)
12	Process item i	-
-	item_idx	2
-	weight	2

Bước	Mô tả / Biến	Giá trị
-	value	3
-	dp_row	(0, 2, 3, 5, 5)
13	Process item i	-
-	item_idx	3
-	weight	3
-	value	4
-	dp_row	(0, 2, 3, 5, 6)
99	Found optimal Knapsack solution	-
-	max_value	6
-	weight_used	4

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	3
-	W	8
11	Process item i	-
-	item_idx	1
-	weight	3
-	value	30
-	dp_row	(0, 0, 0, 30, 30, 30, 30, 30, 30)
12	Process item i	-
-	item_idx	2
-	weight	4
-	value	50
-	dp_row	(0, 0, 0, 30, 50, 50, 50, 80, 80)
13	Process item i	-
-	item_idx	3
-	weight	5
-	value	60
-	dp_row	(0, 0, 0, 30, 50, 60, 60, 80, 90)
99	Found optimal Knapsack solution	-
-	max_value	90
-	weight_used	8

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	1
-	W	10
11	Process item i	-
-	item_idx	1
-	weight	15

Bước	Mô tả / Biến	Giá trị
-	value	100
-	dp_row	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
99	Found optimal Knapsack solution	-
-	max_value	0
-	weight_used	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	1
-	W	5
11	Process item i	-
-	item_idx	1
-	weight	5
-	value	20
-	dp_row	(0, 0, 0, 0, 0, 20)
99	Found optimal Knapsack solution	-
-	max_value	20
-	weight_used	5

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	3
-	W	1
11	Process item i	-
-	item_idx	1
-	weight	2
-	value	10
-	dp_row	(0, 0)
12	Process item i	-
-	item_idx	2
-	weight	3
-	value	20
-	dp_row	(0, 0)
13	Process item i	-
-	item_idx	3
-	weight	4
-	value	30
-	dp_row	(0, 0)
99	Found optimal Knapsack solution	-
-	max_value	0
-	weight_used	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	4
-	W	10
11	Process item i	-
-	item_idx	1
-	weight	5
-	value	10
-	dp_row	(0, 0, 0, 0, 0, 10, 10, 10, 10, 10, 10)
12	Process item i	-
-	item_idx	2
-	weight	5
-	value	10
-	dp_row	(0, 0, 0, 0, 0, 10, 10, 10, 10, 10, 20)
13	Process item i	-
-	item_idx	3
-	weight	5
-	value	10
-	dp_row	(0, 0, 0, 0, 0, 10, 10, 10, 10, 10, 20)
14	Process item i	-
-	item_idx	4
-	weight	5
-	value	10
-	dp_row	(0, 0, 0, 0, 0, 10, 10, 10, 10, 10, 20)
99	Found optimal Knapsack solution	-
-	max_value	20
-	weight_used	10

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	3
-	W	10
11	Process item i	-
-	item_idx	1
-	weight	2
-	value	50
-	dp_row	(0, 0, 50, 50, 50, 50, 50, 50, 50, 50, 50)
12	Process item i	-
-	item_idx	2
-	weight	8
-	value	1
-	dp_row	(0, 0, 50, 50, 50, 50, 50, 50, 50, 50, 51)

Bước	Mô tả / Biến	Giá trị
13	Process item i	-
-	item_idx	3
-	weight	10
-	value	50
-	dp_row	(0, 0, 50, 50, 50, 50, 50, 50, 50, 50, 51)
99	Found optimal Knapsack solution	-
-	max_value	51
-	weight_used	10

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	5
-	W	10
11	Process item i	-
-	item_idx	1
-	weight	2
-	value	6
-	dp_row	(0, 0, 6, 6, 6, 6, 6, 6, 6, 6, 6)
12	Process item i	-
-	item_idx	2
-	weight	2
-	value	10
-	dp_row	(0, 0, 10, 10, 16, 16, 16, 16, 16, 16, 16)
13	Process item i	-
-	item_idx	3
-	weight	3
-	value	12
-	dp_row	(0, 0, 10, 12, 16, 22, 22, 28, 28, 28, 28)
14	Process item i	-
-	item_idx	4
-	weight	4
-	value	16
-	dp_row	(0, 0, 10, 12, 16, 22, 26, 28, 32, 38, 38)
15	Process item i	-
-	item_idx	5
-	weight	5
-	value	22
-	dp_row	(0, 0, 10, 12, 16, 22, 26, 32, 34, 38, 44)
99	Found optimal Knapsack solution	-
-	max_value	44
-	weight_used	10

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, W and items	-
-	n	6
-	W	12
11	Process item i	-
-	item_idx	1
-	weight	1
-	value	2
-	dp_row	(0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2)
12	Process item i	-
-	item_idx	2
-	weight	3
-	value	8
-	dp_row	(0, 2, 2, 8, 10, 10, 10, 10, 10, 10, 10, 10)
13	Process item i	-
-	item_idx	3
-	weight	4
-	value	10
-	dp_row	(0, 2, 2, 8, 10, 12, 12, 18, 20, 20, 20, 20)
14	Process item i	-
-	item_idx	4
-	weight	5
-	value	15
-	dp_row	(0, 2, 2, 8, 10, 15, 17, 18, 23, 25, 27, 27, 33)
15	Process item i	-
-	item_idx	5
-	weight	7
-	value	20
-	dp_row	(0, 2, 2, 8, 10, 15, 17, 20, 23, 25, 28, 30, 35)
16	Process item i	-
-	item_idx	6
-	weight	2
-	value	5
-	dp_row	(0, 2, 5, 8, 10, 15, 17, 20, 23, 25, 28, 30, 35)
99	Found optimal Knapsack solution	-
-	max_value	35
-	weight_used	12

Lời Giải Chi Tiết Chương 20: Lý Thuyết Đồ Thị & Cách Biểu Diễn

Lời giải & Đáp án: Bài 20.1 - Biểu Diễn Đồ Thị & Phân Tích Bậc Đỉnh			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	4 3 1 2 2 3 3 4	2 2 0 1	Đường đi 4 đỉnh 3 cạnh
2	5 4 1 2 1 3 1 4 1 5	4 1 0 1	Đồ thị hình sao tâm tại đỉnh 1
3	3 3 1 2 2 3 3 1	2 1 0 1	Đồ thị tam giác chu trình 3 đỉnh
4	1 0	0 1 1 1	Đồ thị chỉ gồm 1 đỉnh cô lập không cạnh
5	4 0	0 1 4 1	Đồ thị không có cạnh nào gồm 4 đỉnh cô lập
6	2 1 1 1	2 1 1 0	Đồ thị có cạnh khuyên tại đỉnh 1
7	3 2 1 2 1 2	2 1 1 0	Đa cạnh song song nối đỉnh 1 và đỉnh 2
8	5 4 1 2 2 1 3 3 4 5	2 1 0 0	Đồ thị hỗn hợp vừa chứa đa cạnh vừa chứa cạnh khuyên
9	6 5 1 2 2 3 4 5 5 6 4 6	2 2 0 1	Hai thành phần liên thông riêng biệt không có đỉnh cô lập
10	7 6 1 2 1 3 1 4 2 3 5 6 7 7	3 1 0 0	Đồ thị 7 đỉnh có cấu trúc bánh xe kết hợp khuyên

Bảng Vết Thực Thi Từng Bước (Chương 20)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-

Bước	Mô tả / Biến	Giá trị
-	n	4
-	m	3
2	Degrees analyzed	-
-	max_deg	2
-	best_vertex	2
-	isolated_count	0
-	is_simple	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-
-	n	5
-	m	4
2	Degrees analyzed	-
-	max_deg	4
-	best_vertex	1
-	isolated_count	0
-	is_simple	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-
-	n	3
-	m	3
2	Degrees analyzed	-
-	max_deg	2
-	best_vertex	1
-	isolated_count	0
-	is_simple	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-
-	n	1
-	m	0
2	Degrees analyzed	-
-	max_deg	0
-	best_vertex	1
-	isolated_count	1
-	is_simple	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-
-	n	4
-	m	0

Bước	Mô tả / Biến	Giá trị
2	Degrees analyzed	-
-	max_deg	0
-	best_vertex	1
-	isolated_count	4
-	is_simple	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-
-	n	2
-	m	1
2	Degrees analyzed	-
-	max_deg	2
-	best_vertex	1
-	isolated_count	1
-	is_simple	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-
-	n	3
-	m	2
2	Degrees analyzed	-
-	max_deg	2
-	best_vertex	1
-	isolated_count	1
-	is_simple	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-
-	n	5
-	m	4
2	Degrees analyzed	-
-	max_deg	2
-	best_vertex	1
-	isolated_count	0
-	is_simple	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-
-	n	6
-	m	5
2	Degrees analyzed	-
-	max_deg	2

Bước	Mô tả / Biến	Giá trị
-	best_vertex	2
-	isolated_count	0
-	is_simple	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m	-
-	n	7
-	m	6
2	Degrees analyzed	-
-	max_deg	3
-	best_vertex	1
-	isolated_count	0
-	is_simple	0

Lời Giải Chi Tiết Chương 21: Duyệt Đồ Thị Cơ Bản BFS & DFS

Lời giải & Đáp án: Bài 21.1 - Duyệt Đồ Thị Cơ Bản: Thành Phần Liên Thông & Đồ Thị Hai Phía			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	6 5 1 2 2 3 3 1 4 5 5 6	2 3 0	Hai thành phần liên thông, một thành phần chứa tam giác lẻ
2	4 3 1 2 2 3 3 4	1 4 1	Đường đi 4 đỉnh liên thông và hai phía
3	4 4 1 2 2 3 3 4 4 1	1 4 1	Chu trình chẵn 4 đỉnh là đồ thị hai phía
4	1 0	1 1 1	Đồ thị 1 đỉnh cô lập
5	5 0	5 1 1	5 đỉnh cô lập không có cạnh
6	2 1 1 2	1 2 1	Một cạnh đơn giữa 2 đỉnh
7	3 3 1 2 2 3 3 1	1 3 0	Đồ thị đầy đủ 3 đỉnh K3 chứa chu trình lẻ
8	5 5 1 2 2 3 3 4 4 5 5 1	1 5 0	Chu trình lẻ C5 ngũ giác không phải hai phía
9	7 6 1 2 2 3 3 4 5 6 6 7 5 7	2 4 0	Thử thách tính nhẩm 7 đỉnh gồm đường đi và tam giác
10	8 6 1 2 3 4 5 6 7 8 1 3 5 7	2 4 1	Thử thách tính nhẩm 8 đỉnh gồm hai cây 4 đỉnh rời nhau

Bảng Vết Thực Thi Từng Bước (Chương 21)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	6
-	m	5
11	Start new component BFS	-
-	start_vertex	1
-	colored_vertex	2
-	assigned_color	1
-	colored_vertex	3
-	assigned_color	1
999	Conflict found: odd cycle detected	-
-	conflict_u	2
-	conflict_v	3
999	Conflict found: odd cycle detected	-
-	conflict_u	3
-	conflict_v	2
-	current_component_size	3
12	Start new component BFS	-
-	start_vertex	4
-	colored_vertex	5
-	assigned_color	1
-	colored_vertex	6
-	assigned_color	0
-	current_component_size	3
100	Graph analysis complete	-
-	comp_count	2
-	max_comp_size	3
-	is_bipartite	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	4
-	m	3
11	Start new component BFS	-
-	start_vertex	1
-	colored_vertex	2
-	assigned_color	1
-	colored_vertex	3
-	assigned_color	0
-	colored_vertex	4

Bước	Mô tả / Biến	Giá trị
-	assigned_color	1
-	current_component_size	4
100	Graph analysis complete	-
-	comp_count	1
-	max_comp_size	4
-	is_bipartite	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	4
-	m	4
11	Start new component BFS	-
-	start_vertex	1
-	colored_vertex	2
-	assigned_color	1
-	colored_vertex	4
-	assigned_color	1
-	colored_vertex	3
-	assigned_color	0
-	current_component_size	4
100	Graph analysis complete	-
-	comp_count	1
-	max_comp_size	4
-	is_bipartite	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	1
-	m	0
11	Start new component BFS	-
-	start_vertex	1
-	current_component_size	1
100	Graph analysis complete	-
-	comp_count	1
-	max_comp_size	1
-	is_bipartite	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	5
-	m	0
11	Start new component BFS	-

Bước	Mô tả / Biến	Giá trị
-	start_vertex	1
-	current_component_size	1
12	Start new component BFS	-
-	start_vertex	2
-	current_component_size	1
13	Start new component BFS	-
-	start_vertex	3
-	current_component_size	1
14	Start new component BFS	-
-	start_vertex	4
-	current_component_size	1
15	Start new component BFS	-
-	start_vertex	5
-	current_component_size	1
100	Graph analysis complete	-
-	comp_count	5
-	max_comp_size	1
-	is_bipartite	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	2
-	m	1
11	Start new component BFS	-
-	start_vertex	1
-	colored_vertex	2
-	assigned_color	1
-	current_component_size	2
100	Graph analysis complete	-
-	comp_count	1
-	max_comp_size	2
-	is_bipartite	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	3
-	m	3
11	Start new component BFS	-
-	start_vertex	1
-	colored_vertex	2
-	assigned_color	1

Bước	Mô tả / Biến	Giá trị
-	colored_vertex	3
-	assigned_color	1
999	Conflict found: odd cycle detected	-
-	conflict_u	2
-	conflict_v	3
999	Conflict found: odd cycle detected	-
-	conflict_u	3
-	conflict_v	2
-	current_component_size	3
100	Graph analysis complete	-
-	comp_count	1
-	max_comp_size	3
-	is_bipartite	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	5
-	m	5
11	Start new component BFS	-
-	start_vertex	1
-	colored_vertex	2
-	assigned_color	1
-	colored_vertex	5
-	assigned_color	1
-	colored_vertex	3
-	assigned_color	0
-	colored_vertex	4
-	assigned_color	0
999	Conflict found: odd cycle detected	-
-	conflict_u	3
-	conflict_v	4
999	Conflict found: odd cycle detected	-
-	conflict_u	4
-	conflict_v	3
-	current_component_size	5
100	Graph analysis complete	-
-	comp_count	1
-	max_comp_size	5
-	is_bipartite	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	7
-	m	6
11	Start new component BFS	-
-	start_vertex	1
-	colored_vertex	2
-	assigned_color	1
-	colored_vertex	3
-	assigned_color	0
-	colored_vertex	4
-	assigned_color	1
-	current_component_size	4
12	Start new component BFS	-
-	start_vertex	5
-	colored_vertex	6
-	assigned_color	1
-	colored_vertex	7
-	assigned_color	1
999	Conflict found: odd cycle detected	-
-	conflict_u	6
-	conflict_v	7
999	Conflict found: odd cycle detected	-
-	conflict_u	7
-	conflict_v	6
-	current_component_size	3
100	Graph analysis complete	-
-	comp_count	2
-	max_comp_size	4
-	is_bipartite	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and adjacency list	-
-	n	8
-	m	6
11	Start new component BFS	-
-	start_vertex	1
-	colored_vertex	2
-	assigned_color	1
-	colored_vertex	3
-	assigned_color	1
-	colored_vertex	4

Bước	Mô tả / Biến	Giá trị
-	assigned_color	0
-	current_component_size	4
12	Start new component BFS	-
-	start_vertex	5
-	colored_vertex	6
-	assigned_color	1
-	colored_vertex	7
-	assigned_color	1
-	colored_vertex	8
-	assigned_color	0
-	current_component_size	4
100	Graph analysis complete	-
-	comp_count	2
-	max_comp_size	4
-	is_bipartite	1

Lời Giải Chi Tiết Chương 22: Đồ Thị Không Chu Trình (DAG) & Sắp Xếp Tô-pô

Lời giải & Đáp án: Bài 22.1 - Đồ Thị Không Chu Trình (DAG): Sắp Xếp Tô-pô & Đường Đi Dài Nhất			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	4 4 1 2 1 3 2 4 3 4	2 1 2 3 4	Đồ thị kim cương 4 đỉnh không chu trình
2	3 2 1 2 2 3	2 1 2 3	Đoạn đường thẳng có hướng 3 đỉnh
3	4 3 4 3 3 2 2 1	3 4 3 2 1	Đoạn đường thẳng thứ tự ngược 4->3->2->1
4	1 0	0 1	Đồ thị 1 đỉnh không có cạnh
5	3 0	0 1 2 3	3 đỉnh rời nhau không có cạnh
6	2 1 1 2	1 1 2	Một cung đơn từ 1 sang 2
7	3 3 1 2 2 3 3 1	-1	Đồ thị có chu trình có hướng 3 đỉnh
8	4 4 1 2 2 3 3 2 3 4	-1	Đồ thị có chu trình cục bộ giữa hai đỉnh 2 và 3
9	5 5 1 4 2 4 3 5 4 5 1 2	3 1 2 3 4 5	Thử thách tính nhẩm thứ tự tô-pô 5 đỉnh và đường đi dài nhất
10	6 7 1 2 1 3 2 4 3 4 4 5 4 6 5 6	4 1 2 3 4 5 6	Thử thách tính nhẩm DAG 6 đỉnh đường đi dài nhất 4 cạnh

Bảng Vết Thực Thi Từng Bước (Chương 22)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-
-	n	4
-	m	4
-	in_degree	(0, 0, 1, 1, 2)
11	Pop vertex in Kahn algorithm	-
-	popped_vertex	1
12	Pop vertex in Kahn algorithm	-
-	popped_vertex	2
13	Pop vertex in Kahn algorithm	-
-	popped_vertex	3
14	Pop vertex in Kahn algorithm	-
-	popped_vertex	4
100	Topological sort & DP complete	-
-	longest_path	2
-	topo_order	(1, 2, 3, 4)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-
-	n	3
-	m	2
-	in_degree	(0, 0, 1, 1)
11	Pop vertex in Kahn algorithm	-
-	popped_vertex	1
12	Pop vertex in Kahn algorithm	-
-	popped_vertex	2
13	Pop vertex in Kahn algorithm	-
-	popped_vertex	3
100	Topological sort & DP complete	-
-	longest_path	2
-	topo_order	(1, 2, 3)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-
-	n	4
-	m	3
-	in_degree	(0, 1, 1, 1, 0)
11	Pop vertex in Kahn algorithm	-

Bước	Mô tả / Biến	Giá trị
-	popped_vertex	4
12	Pop vertex in Kahn algorithm	-
-	popped_vertex	3
13	Pop vertex in Kahn algorithm	-
-	popped_vertex	2
14	Pop vertex in Kahn algorithm	-
-	popped_vertex	1
100	Topological sort & DP complete	-
-	longest_path	3
-	topo_order	(4, 3, 2, 1)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-
-	n	1
-	m	0
-	in_degree	(0, 0)
11	Pop vertex in Kahn algorithm	-
-	popped_vertex	1
100	Topological sort & DP complete	-
-	longest_path	0
-	topo_order	(1,)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-
-	n	3
-	m	0
-	in_degree	(0, 0, 0, 0)
11	Pop vertex in Kahn algorithm	-
-	popped_vertex	1
12	Pop vertex in Kahn algorithm	-
-	popped_vertex	2
13	Pop vertex in Kahn algorithm	-
-	popped_vertex	3
100	Topological sort & DP complete	-
-	longest_path	0
-	topo_order	(1, 2, 3)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-

Bước	Mô tả / Biến	Giá trị
-	n	2
-	m	1
-	in_degree	(0, 0, 1)
11	Pop vertex in Kahn algorithm	-
-	popped_vertex	1
12	Pop vertex in Kahn algorithm	-
-	popped_vertex	2
100	Topological sort & DP complete	-
-	longest_path	1
-	topo_order	(1, 2)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-
-	n	3
-	m	3
-	in_degree	(0, 1, 1, 1)
999	Cycle detected, not a DAG	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-
-	n	4
-	m	4
-	in_degree	(0, 0, 2, 1, 1)
11	Pop vertex in Kahn algorithm	-
-	popped_vertex	1
999	Cycle detected, not a DAG	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-
-	n	5
-	m	5
-	in_degree	(0, 0, 1, 0, 2, 2)
11	Pop vertex in Kahn algorithm	-
-	popped_vertex	1
12	Pop vertex in Kahn algorithm	-
-	popped_vertex	2
13	Pop vertex in Kahn algorithm	-
-	popped_vertex	3
14	Pop vertex in Kahn algorithm	-

Bước	Mô tả / Biến	Giá trị
-	popped_vertex	4
15	Pop vertex in Kahn algorithm	-
-	popped_vertex	5
100	Topological sort & DP complete	-
-	longest_path	3
-	topo_order	(1, 2, 3, 4, 5)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and constructed DAG in-degrees	-
-	n	6
-	m	7
-	in_degree	(0, 0, 1, 1, 2, 1, 2)
11	Pop vertex in Kahn algorithm	-
-	popped_vertex	1
12	Pop vertex in Kahn algorithm	-
-	popped_vertex	2
13	Pop vertex in Kahn algorithm	-
-	popped_vertex	3
14	Pop vertex in Kahn algorithm	-
-	popped_vertex	4
15	Pop vertex in Kahn algorithm	-
-	popped_vertex	5
16	Pop vertex in Kahn algorithm	-
-	popped_vertex	6
100	Topological sort & DP complete	-
-	longest_path	4
-	topo_order	(1, 2, 3, 4, 5, 6)

Lời Giải Chi Tiết Chương 23: Cây (Trees) & Các Thuộc Tính Đặc Biệt

Lời giải & Đáp án: Bài 23.1 - Cây & Thuộc Tính Cốt Lõi: Chiều Cao, Đường Kính & Cây Con			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	5 1 2 1 3 2 4 2 5	3 2 3 3	Cây nhị phân mẫu 5 đỉnh
2	4 1 2 2 3 3 4	3 3 1 3	Cây dạng đường thẳng 4 đỉnh
3	5 1 2 1 3 1 4 1 5	2 1 4 1	Cây hình sao 5 đỉnh tâm tại gốc 1
4	1	0 0 1 0	Cây chỉ gồm 1 đỉnh duy nhất
5	2 1 2	1 1 1 1	Cây chỉ gồm 2 đỉnh 1 cạnh
6	3 1 2 1 3	2 1 2 1	Cây 3 đỉnh hình chữ V có gốc tại 1
7	6 1 2 2 3 3 4 4 5 5 6	5 5 1 5	Cây suy biến thành chuỗi thẳng 6 đỉnh
8	6 2 1 2 3 2 4 2 5 2 6	2 2 4 5	Cây hình sao tâm tại đỉnh 2 khác gốc 1
9	7 1 2 1 3 2 4 2 5 3 6 3 7	4 2 4 3	Cây nhị phân cân bằng hoàn chỉnh 7 đỉnh

Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
10	8 1 2 2 3 3 4 1 5 5 6 6 7 7 8	7 4 2 4	Cây 8 đỉnh phân hai nhánh không đối xứng

Bảng Vết Thực Thi Từng Bước (Chương 23)

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	5
2	First DFS finished	-
-	node_a	4
3	Second DFS finished	-
-	node_b	3
-	diameter	3
4	Tree metrics calculated	-
-	diameter	3
-	max_depth	2
-	leaf_count	3
-	max_child_subtree_size	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	4
2	First DFS finished	-
-	node_a	4
3	Second DFS finished	-
-	node_b	1
-	diameter	3
4	Tree metrics calculated	-
-	diameter	3
-	max_depth	3
-	leaf_count	1
-	max_child_subtree_size	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	5
2	First DFS finished	-
-	node_a	2

Bước	Mô tả / Biến	Giá trị
3	Second DFS finished	-
-	node_b	3
-	diameter	2
4	Tree metrics calculated	-
-	diameter	2
-	max_depth	1
-	leaf_count	4
-	max_child_subtree_size	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	1
2	Single node tree	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	2
2	First DFS finished	-
-	node_a	2
3	Second DFS finished	-
-	node_b	1
-	diameter	1
4	Tree metrics calculated	-
-	diameter	1
-	max_depth	1
-	leaf_count	1
-	max_child_subtree_size	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	3
2	First DFS finished	-
-	node_a	2
3	Second DFS finished	-
-	node_b	3
-	diameter	2
4	Tree metrics calculated	-
-	diameter	2
-	max_depth	1
-	leaf_count	2
-	max_child_subtree_size	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	6
2	First DFS finished	-
-	node_a	6
3	Second DFS finished	-
-	node_b	1
-	diameter	5
4	Tree metrics calculated	-
-	diameter	5
-	max_depth	5
-	leaf_count	1
-	max_child_subtree_size	5

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	6
2	First DFS finished	-
-	node_a	3
3	Second DFS finished	-
-	node_b	1
-	diameter	2
4	Tree metrics calculated	-
-	diameter	2
-	max_depth	2
-	leaf_count	4
-	max_child_subtree_size	5

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	7
2	First DFS finished	-
-	node_a	4
3	Second DFS finished	-
-	node_b	6
-	diameter	4
4	Tree metrics calculated	-
-	diameter	4
-	max_depth	2
-	leaf_count	4
-	max_child_subtree_size	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree adjacency list	-
-	n	8
2	First DFS finished	-
-	node_a	8
3	Second DFS finished	-
-	node_b	4
-	diameter	7
4	Tree metrics calculated	-
-	diameter	7
-	max_depth	4
-	leaf_count	2
-	max_child_subtree_size	4

Lời Giải Chi Tiết Chương 24: Đường Đi Ngắn Nhất Với Thuật Toán Dijkstra

Lời giải & Đáp án: Bài 24.1 - Thuật Toán Dijkstra Chuẩn Mục & Phục Hồi Đường Đi			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	4 4 1 2 2 1 3 5 2 4 4 3 4 1	6 2 2	Đồ thị 4 đỉnh hai nhánh đường đi có cùng độ dài
2	3 3 1 2 10 2 3 5 1 3 20	15 2 2	Đường đi qua đỉnh trung gian ngắn hơn cạnh trực tiếp
3	4 3 1 2 3 2 3 4 3 4 5	12 3 3	Đường đi thẳng nối tiếp 4 đỉnh
4	1 0	0 0 -1	Đồ thị 1 đỉnh nguồn trùng đích
5	3 1 1 2 5	-1	Đỉnh đích không thể tiếp cận
6	2 1 1 2 0	0 1 1	Cung trực tiếp có trọng số bằng 0
7	3 2 2 3 4 3 1 2	-1	Chu trình ngược không có đường đi từ đỉnh 1
8	4 4 1 2 10 1 3 1 3 2 1 2 4 1	3 3 2	Đường đi vòng qua nhiều đỉnh trung gian tối ưu hơn
9	5 6 1 2 4 1 3 2 3 2 1 2 4 5 3 5 8 4 5 1	9 4 4	Thử thách tính nhẩm 5 đỉnh qua 4 chặng đường
10	6 8 1 2 3 1 3 5 2 4 2 3 4 1 2 5 6 4 5 2 4 6 7 5 6 1	8 4 5	Thử thách tính nhẩm 6 đỉnh lựa chọn rẽ nhánh tối ưu

Bảng Vết Thực Thi Từng Bước (Chương 24)

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	4
-	m	4
11	Extract min vertex u	-
-	u	1
-	d	0
101	Relax edge u -> v	-
-	v	2
-	new_dist	2
-	parent	1
101	Relax edge u -> v	-
-	v	3
-	new_dist	5
-	parent	1
12	Extract min vertex u	-
-	u	2
-	d	2
102	Relax edge u -> v	-
-	v	4
-	new_dist	6
-	parent	2
13	Extract min vertex u	-
-	u	3
-	d	5
14	Extract min vertex u	-
-	u	4
-	d	6
999	Shortest path to n found	-
-	shortest_dist	6
-	edge_count	2
-	parent_n	2

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	3
-	m	3
11	Extract min vertex u	-
-	u	1
-	d	0

Bước	Mô tả / Biến	Giá trị
101	Relax edge $u \rightarrow v$	-
-	v	2
-	new_dist	10
-	parent	1
101	Relax edge $u \rightarrow v$	-
-	v	3
-	new_dist	20
-	parent	1
12	Extract min vertex u	-
-	u	2
-	d	10
102	Relax edge $u \rightarrow v$	-
-	v	3
-	new_dist	15
-	parent	2
13	Extract min vertex u	-
-	u	3
-	d	15
999	Shortest path to n found	-
-	shortest_dist	15
-	edge_count	2
-	parent_n	2

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	4
-	m	3
11	Extract min vertex u	-
-	u	1
-	d	0
101	Relax edge $u \rightarrow v$	-
-	v	2
-	new_dist	3
-	parent	1
12	Extract min vertex u	-
-	u	2
-	d	3
102	Relax edge $u \rightarrow v$	-
-	v	3
-	new_dist	7
-	parent	2

Bước	Mô tả / Biến	Giá trị
13	Extract min vertex u	-
-	u	3
-	d	7
103	Relax edge u -> v	-
-	v	4
-	new_dist	12
-	parent	3
14	Extract min vertex u	-
-	u	4
-	d	12
999	Shortest path to n found	-
-	shortest_dist	12
-	edge_count	3
-	parent_n	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	1
-	m	0
11	Extract min vertex u	-
-	u	1
-	d	0
999	Shortest path to n found	-
-	shortest_dist	0
-	edge_count	0
-	parent_n	-1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	3
-	m	1
11	Extract min vertex u	-
-	u	1
-	d	0
101	Relax edge u -> v	-
-	v	2
-	new_dist	5
-	parent	1
12	Extract min vertex u	-
-	u	2
-	d	5

Bước	Mô tả / Biến	Giá trị
999	Destination unreachable	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	2
-	m	1
11	Extract min vertex u	-
-	u	1
-	d	0
101	Relax edge u -> v	-
-	v	2
-	new_dist	0
-	parent	1
12	Extract min vertex u	-
-	u	2
-	d	0
999	Shortest path to n found	-
-	shortest_dist	0
-	edge_count	1
-	parent_n	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	3
-	m	2
11	Extract min vertex u	-
-	u	1
-	d	0
999	Destination unreachable	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	4
-	m	4
11	Extract min vertex u	-
-	u	1
-	d	0
101	Relax edge u -> v	-
-	v	2
-	new_dist	10
-	parent	1
101	Relax edge u -> v	-

Bước	Mô tả / Biến	Giá trị
-	v	3
-	new_dist	1
-	parent	1
12	Extract min vertex u	-
-	u	3
-	d	1
102	Relax edge u -> v	-
-	v	2
-	new_dist	2
-	parent	3
13	Extract min vertex u	-
-	u	2
-	d	2
103	Relax edge u -> v	-
-	v	4
-	new_dist	3
-	parent	2
14	Extract min vertex u	-
-	u	4
-	d	3
999	Shortest path to n found	-
-	shortest_dist	3
-	edge_count	3
-	parent_n	2

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	5
-	m	6
11	Extract min vertex u	-
-	u	1
-	d	0
101	Relax edge u -> v	-
-	v	2
-	new_dist	4
-	parent	1
101	Relax edge u -> v	-
-	v	3
-	new_dist	2
-	parent	1
12	Extract min vertex u	-

Bước	Mô tả / Biến	Giá trị
-	u	3
-	d	2
102	Relax edge u -> v	-
-	v	2
-	new_dist	3
-	parent	3
102	Relax edge u -> v	-
-	v	5
-	new_dist	10
-	parent	3
13	Extract min vertex u	-
-	u	2
-	d	3
103	Relax edge u -> v	-
-	v	4
-	new_dist	8
-	parent	2
14	Extract min vertex u	-
-	u	4
-	d	8
104	Relax edge u -> v	-
-	v	5
-	new_dist	9
-	parent	4
15	Extract min vertex u	-
-	u	5
-	d	9
999	Shortest path to n found	-
-	shortest_dist	9
-	edge_count	4
-	parent_n	4

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	6
-	m	8
11	Extract min vertex u	-
-	u	1
-	d	0
101	Relax edge u -> v	-
-	v	2

Bước	Mô tả / Biến	Giá trị
-	new_dist	3
-	parent	1
101	Relax edge u -> v	-
-	v	3
-	new_dist	5
-	parent	1
12	Extract min vertex u	-
-	u	2
-	d	3
102	Relax edge u -> v	-
-	v	4
-	new_dist	5
-	parent	2
102	Relax edge u -> v	-
-	v	5
-	new_dist	9
-	parent	2
13	Extract min vertex u	-
-	u	3
-	d	5
14	Extract min vertex u	-
-	u	4
-	d	5
104	Relax edge u -> v	-
-	v	5
-	new_dist	7
-	parent	4
104	Relax edge u -> v	-
-	v	6
-	new_dist	12
-	parent	4
15	Extract min vertex u	-
-	u	5
-	d	7
105	Relax edge u -> v	-
-	v	6
-	new_dist	8
-	parent	5
16	Extract min vertex u	-
-	u	6
-	d	8

Bước	Mô tả / Biến	Giá trị
999	Shortest path to n found	-
-	shortest_dist	8
-	edge_count	4
-	parent_n	5

Lời Giải Chi Tiết Bài Tập Tích Hợp: Đồ Thị, Dijkstra & Quy Hoạch Động Trên DAG

Lời giải & Đáp án: Bài Tập Tích Hợp 3 - Lộ Trình Tối Ưu: Dijkstra & Quy Hoạch Động Trên Shortest Path DAG			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	4 5 1 2 2 1 3 3 2 4 3 3 4 2 1 4 5	5 3 1 2	Đồ thị mẫu 4 đỉnh 3 đường đi ngắn nhất cùng độ dài 5
2	3 3 1 2 4 2 3 6 1 3 10	10 2 1 2	Tam giác 3 đỉnh có 2 đường đi cùng độ dài 10
3	4 3 1 2 1 2 3 2 3 4 3	6 1 3 3	Đoạn đường thẳng duy nhất 4 đỉnh
4	1 0	0 1 0 0	Đồ thị 1 đỉnh nguồn trùng đích
5	3 1 1 2 5	-1	Đỉnh đích không thể tiếp cận
6	2 1 1 2 100	100 1 1 1	Một cạnh trực tiếp duy nhất từ 1 đến 2
7	4 4 1 2 5 2 3 5 3 4 5 1 4 20	15 1 3 3	Đường trực tiếp dài hơn đường vòng nhiều chặng
8	3 2 2 3 1 3 1 1	-1	Đồ thị có chu trình nhưng đỉnh 1 không có cạnh ra
9	5 7 1 2 1 1 3 1 2 4 2 3 4 2 2 5 5 3 5 5 4 5 2	5 2 3 3	Thử thách tính nhẩm 5 đỉnh hai đường đi ngắn nhất

Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
10	6 9 1 2 2 1 3 2 2 4 3 3 4 3 4 5 1 2 5 4 3 5 4 5 6 2 4 6 3	8 6 3 4	Thử thách tính nhẩm 6 đỉnh mạng lưới đa đường đi tối ưu

Bảng Vết Thực Thi Từng Bước (Bài Tập Tích Hợp)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	4
-	m	5
2	Dijkstra phase completed	-
-	dist_n	5
11	Process node u in DAG DP	-
-	u	1
-	dist_u	0
-	ways_u	1
12	Process node u in DAG DP	-
-	u	2
-	dist_u	2
-	ways_u	1
13	Process node u in DAG DP	-
-	u	3
-	dist_u	3
-	ways_u	1
14	Process node u in DAG DP	-
-	u	4
-	dist_u	5
-	ways_u	3
100	Integrated solution computed	-
-	min_dist	5
-	ways	3
-	min_edges	1
-	max_edges	2

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	3

Bước	Mô tả / Biến	Giá trị
-	m	3
2	Dijkstra phase completed	-
-	dist_n	10
11	Process node u in DAG DP	-
-	u	1
-	dist_u	0
-	ways_u	1
12	Process node u in DAG DP	-
-	u	2
-	dist_u	4
-	ways_u	1
13	Process node u in DAG DP	-
-	u	3
-	dist_u	10
-	ways_u	2
100	Integrated solution computed	-
-	min_dist	10
-	ways	2
-	min_edges	1
-	max_edges	2

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	4
-	m	3
2	Dijkstra phase completed	-
-	dist_n	6
11	Process node u in DAG DP	-
-	u	1
-	dist_u	0
-	ways_u	1
12	Process node u in DAG DP	-
-	u	2
-	dist_u	1
-	ways_u	1
13	Process node u in DAG DP	-
-	u	3
-	dist_u	3
-	ways_u	1
14	Process node u in DAG DP	-
-	u	4

Bước	Mô tả / Biến	Giá trị
-	dist_u	6
-	ways_u	1
100	Integrated solution computed	-
-	min_dist	6
-	ways	1
-	min_edges	3
-	max_edges	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	1
-	m	0
2	Dijkstra phase completed	-
-	dist_n	0
11	Process node u in DAG DP	-
-	u	1
-	dist_u	0
-	ways_u	1
100	Integrated solution computed	-
-	min_dist	0
-	ways	1
-	min_edges	0
-	max_edges	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	3
-	m	1
2	Dijkstra phase completed	-
-	dist_n	1000000000000000000
999	Node n unreachable	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	2
-	m	1
2	Dijkstra phase completed	-
-	dist_n	100
11	Process node u in DAG DP	-
-	u	1
-	dist_u	0
-	ways_u	1

Bước	Mô tả / Biến	Giá trị
12	Process node u in DAG DP	-
-	u	2
-	dist_u	100
-	ways_u	1
100	Integrated solution computed	-
-	min_dist	100
-	ways	1
-	min_edges	1
-	max_edges	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	4
-	m	4
2	Dijkstra phase completed	-
-	dist_n	15
11	Process node u in DAG DP	-
-	u	1
-	dist_u	0
-	ways_u	1
12	Process node u in DAG DP	-
-	u	2
-	dist_u	5
-	ways_u	1
13	Process node u in DAG DP	-
-	u	3
-	dist_u	10
-	ways_u	1
14	Process node u in DAG DP	-
-	u	4
-	dist_u	15
-	ways_u	1
100	Integrated solution computed	-
-	min_dist	15
-	ways	1
-	min_edges	3
-	max_edges	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	3

Bước	Mô tả / Biến	Giá trị
-	m	2
2	Dijkstra phase completed	-
-	dist_n	1000000000000000000
999	Node n unreachable	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	5
-	m	7
2	Dijkstra phase completed	-
-	dist_n	5
11	Process node u in DAG DP	-
-	u	1
-	dist_u	0
-	ways_u	1
12	Process node u in DAG DP	-
-	u	2
-	dist_u	1
-	ways_u	1
13	Process node u in DAG DP	-
-	u	3
-	dist_u	1
-	ways_u	1
14	Process node u in DAG DP	-
-	u	4
-	dist_u	3
-	ways_u	2
15	Process node u in DAG DP	-
-	u	5
-	dist_u	5
-	ways_u	2
100	Integrated solution computed	-
-	min_dist	5
-	ways	2
-	min_edges	3
-	max_edges	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, m and graph	-
-	n	6
-	m	9

Bước	Mô tả / Biến	Giá trị
2	Dijkstra phase completed	-
-	dist_n	8
11	Process node u in DAG DP	-
-	u	1
-	dist_u	0
-	ways_u	1
12	Process node u in DAG DP	-
-	u	2
-	dist_u	2
-	ways_u	1
13	Process node u in DAG DP	-
-	u	3
-	dist_u	2
-	ways_u	1
14	Process node u in DAG DP	-
-	u	4
-	dist_u	5
-	ways_u	2
15	Process node u in DAG DP	-
-	u	5
-	dist_u	6
-	ways_u	4
16	Process node u in DAG DP	-
-	u	6
-	dist_u	8
-	ways_u	6
100	Integrated solution computed	-
-	min_dist	8
-	ways	6
-	min_edges	3
-	max_edges	4